



Meu **AU**migo

Documentação técnica do sistema

Não é um sistema para controlar doações. É um sistema para mostrar ao padrinho o impacto que o dinheiro dele está causando na vida de um animal específico.

VERSÃO

1.0.0

DATA

22 de setembro de 2026

PLATAFORMA

**Next.js 15 · Prisma ·
PostgreSQL · Docker**

LINHAS DE CÓDIGO

15.454

PORTA PUBLICADA

1046

CONTAINER

meuaumigo-app

Sumário

1 Conceito e princípio central

- 1.1 O ciclo que o sistema gerencia
- 1.2 As três áreas
- 1.3 Escopo negativo: o que o sistema deliberadamente não faz

2 Mapa de telas

- 2.1 Público — sem autenticação
- 2.2 Área do padrinho
- 2.3 Administração da ONG
- 2.4 Rotas de API

3 Visão geral

4 Stack e a razão de cada escolha

- 4.1 O que foi deixado de fora, de propósito

5 Estrutura de diretórios

- 5.1 A regra de dependência

6 Ciclo de vida de uma requisição

- 6.1 Leitura — abrir uma página
- 6.2 Escrita — enviar um formulário

7 Visão do modelo

- 7.1 Convenções do schema

8 Dicionário de dados

- 8.1 Pessoas e acesso
- 8.2 Animal e sua história
- 8.3 Saúde
- 8.4 O dinheiro que entra
- 8.5 O dinheiro que sai
- 8.6 Estoque
- 8.7 Campanha, adoção, notificação e configuração

9 Enumerações

10 Migrations e privilégios

11 Custo estimado × gasto real

12 Competência: como o mês fecha

13 O rateio da contribuição

14 Ciclo do apadrinhamento

- 14.1 Nascimento
- 14.2 Cobrança mensal
- 14.3 Pausa e cancelamento

15 Saúde amarrada ao financeiro

16 Estoque: saldo é consequência

17 Funil de adoção

18 Campanhas e reconhecimento

- 18.1 Campanha
- 18.2 Gamificação

19 Dois papéis no banco

20 Sessão e senha

- 20.1 Senha
- 20.2 Token de sessão

21 Defesa de CSRF

22 Autorização por papel

23 Auditoria

24 Mídia, comprovantes e dados pessoais

- 24.1 Arquivos
- 24.2 Foto pública, comprovante condicional
- 24.3 Dados pessoais e a IA

24.4 Superfície pública de escrita

25 **Paleta e tipografia**

26 **Componentes****26.1** Classes de componente**26.2** Peças compartilhadas

27 **Movimento e acessibilidade**

28 **Gráficos****28.1** A paleta categórica, validada**28.2** Anatomia das marcas**28.3** Onde cada forma aparece

29 **O assistente interno****29.1** O que ela não faz**29.2** O contexto enviado**29.3** O prompt de sistema

30 **A imagem**

31 **Compose e a rede**

32 **Entrypoint**

33 **Variáveis de ambiente**

34 **Cabeçalhos e indexação**

35 **A semente****35.1** As duas salvaguardas**35.2** Por que os dados são realistas**35.3** Acessos de demonstração

36 **Deploy****36.1** Instalação em máquina nova

37 **Rotina mensal da equipe**

38 **Backup**

39 **Diagnóstico****39.1** Os três erros recorrentes desta VPS**39.2** Outros diagnósticos**39.3** Onde as coisas estão

A **Referência de arquivos**

B **Comandos****B.1** Container**B.2** Desenvolvimento**B.3** Banco**B.4** Emergência

C **Limitações conhecidas e próximos passos**

D **Glossário**

PARTE I

O produto

1 Conceito e princípio central

O Meu AUmigo é a plataforma de uma ONG de resgate animal. Ela cobre o ciclo inteiro de um animal — do momento em que é recolhido da rua até o dia em que sai adotado — e sustenta financeiramente esse ciclo por meio de **apadrinhamento mensal recorrente**.

Existe uma diferença de propósito que decide praticamente todas as escolhas técnicas deste documento. Um sistema de gestão de doações responde à pergunta *"quanto entrou este mês?"*. Este sistema responde a uma pergunta diferente e muito mais difícil: *"o que os meus cem reais fizeram pelo Thor neste mês?"*.

PRINCÍPIO DE PROJETO

Todo real que entra e todo real que sai precisa conseguir apontar para **um animal**. É essa amarração que permite ao padrinho ver alimentação, medicamento e veterinário discriminados no valor que ele pagou — em vez de receber um agradecimento genérico. Quando uma decisão de modelagem ficou em dúvida, foi este princípio que desempatou.

1.1 O ciclo que o sistema gerencia



O ciclo de vida de um animal no sistema. A seta tracejada é a que sustenta o negócio.

1.2 As três áreas

O sistema é um só aplicativo com três públicos, separados por grupos de rota do Next.js e por papel de usuário:

ÁREA	QUEM USA	O QUE RESOLVE
Site público (publico)	Qualquer visitante, sem login	Vitrine de animais com filtros compartilháveis, perfil de cada animal com história e meta de custeio, campanhas de emergência, formulário de apadrinhamento, formulário de interesse em adoção e a prestação de contas aberta.
Área do padrinho (padrinho)	Quem apadrinha, autenticado	“Como o Thor está?”: última novidade, diário do animal, próxima consulta, tempo de amizade, histórico de pagamentos e o rateio do dinheiro do mês por categoria de gasto.
Administração (ong)	Equipe da ONG, autenticada	Painel com indicadores e gráficos, cadastro e ficha completa dos animais, saúde, alimentação e estoque, financeiro, apadrinhamentos, padrinhos, campanhas, adoções, relatórios, configurações e o assistente de IA.

1.3 Escopo negativo: o que o sistema deliberadamente não faz

Saber o que ficou de fora é tão importante quanto saber o que existe. Cada item abaixo foi uma decisão, não um esquecimento.

NÃO FAZ	POR QUÊ
Não processa pagamento	Não há gateway integrado. Apadrinhamento e doação nascem com status PENDENTE e só viram PAGA quando alguém da equipe confere o extrato. Registrar como pago o que ninguém verificou estragaria justamente o número que a página pública de transparência exhibe.
Não envia e-mail nem push	As notificações são gravadas em tabela e aparecem na área do padrinho. O disparo externo depende de provedor contratado e de domínio verificado — entra quando a ONG tiver os dois.
Não decide nada clínico	O sistema registra o que a veterinária fez; não sugere dose, tratamento ou diagnóstico. A IA é explicitamente proibida de fazê-lo no seu prompt de sistema.
Não aprova adoção automaticamente	O formulário público abre um processo com status INTERESSE . Toda adoção passa por conversa, visita e termo assinado. Adoção por formulário automático é como animal volta para a rua.
Não tem agendador	A VPS não roda cron para este projeto. A geração mensal de cobranças é um botão que a equipe aperta, e por isso foi escrita para ser idempotente.
Não tem modo escuro	O uso real é diurno, no celular, dentro do abrigo. Manter duas paletas validadas custaria mais do que entrega.

2 Mapa de telas

Trinta e uma rotas, todas renderizadas no servidor sob demanda (`dynamic = 'force-dynamic'`), porque quase todas dependem de dados que mudam a cada minuto.

2.1 Público — sem autenticação

ROTA	ARQUIVO	FUNÇÃO
/	(publico)/page.tsx	Abertura com o conceito, campanha urgente em destaque, seis animais sem padrinho, números de impacto e chamada final.
/animais	(publico)/animais/page.tsx	Vitrine com busca e seis grupos de filtro. Os filtros são links, não estado de React.
/animais/[slug]	animais/[slug]/page.tsx	Perfil público: história, galeria, resumo de saúde, diário, caixa de apadrinhamento com a meta, campanhas do animal e mural de padrinhos.
/animais/[slug]/apadrinhar	apadrinhar/page.tsx	Escolha de modalidade, dados do padrinho e confirmação com a chave PIX.
/campanhas	campanhas/page.tsx	Campanhas ativas com barra de meta e as que já deram certo.
/campanhas/[slug]	campanhas/[slug]/page.tsx	Página da campanha com formulário de doação e botão de compartilhar.
/adotar/[slug]	adotar/[slug]/page.tsx	Formulário de interesse em adoção. Recusa educadamente se o animal não estiver disponível.
/impacto	impacto/page.tsx	Transparência: impacto acumulado, entradas × saídas em 12 meses, composição das despesas e comprovantes publicados.
/entrar	(auth)/entrar/page.tsx	Login único para equipe e padrinhos; o destino depende do papel.

2.2 Área do padrinho

ROTA	FUNÇÃO
/meus-aumigos	Cartão por animal apadrinhado com a última novidade, próxima consulta, cobertura do custo e tempo de amizade. Marca as notificações como lidas ao abrir.
/meus-aumigos/[slug]	A tela mais importante do produto: “como o Thor está?”, diário, fotos recentes e o rateio do dinheiro do mês por categoria.
/meus-aumigos/contribuicoes	Histórico completo de pagamentos, com a chave PIX quando há algo em aberto.
/meus-aumigos/conquistas	Tempo de amizade por animal e as conquistas — derivadas, nunca armazenadas.

2.3 Administração da ONG

ROTA	ACESSO	FUNÇÃO
/painel	Equipe	Dez indicadores, alertas acionáveis (sem padrinho, em aberto, estoque baixo), quatro gráficos, agenda de saúde e campanhas.
/animais-ong	Equipe	Grade com busca e cinco filtros; cada cartão traz a cobertura do custo.
/animais-ong/novo /animais-ong/[id]/editar	Equipe	Formulário em quatro blocos, com o custo mensal somando ao vivo.
/animais-ong/[id]	Equipe	Ficha completa e o painel de quatro ações do dia a dia.
/apadrinhamentos	Equipe	Receita recorrente, ticket médio e as ações de pausar, cancelar e reativar.
/padrinhos /padrinhos/[id]	Equipe	Lista e ficha: cadastro, contribuição, conquistas e histórico.
/saude	Equipe	Histórico clínico, agenda de 30 dias e concentração de custo por animal.
/alimentacao	Equipe	Estoque com medidores, previsão de dias e registro de movimento.
/financeiro	Admin	Entradas, saídas, rotina mensal e lançamento de despesa com comprovante.
/campanhas-ong	Equipe	Criação e ciclo de vida das campanhas.
/adocoes	Equipe	Funil de adoção com os próximos passos permitidos.
/relatorios	Equipe	Cobertura, inadimplência, retenção, receita por modalidade e animais descobertos.
/ia	Equipe	Assistente com perguntas prontas; degrada para aviso sem chave de API.
/configuracoes	Admin	Dados da ONG, acessos, veterinários e o registro de auditoria.

2.4 Rotas de API

ROTA	ACESSO	FUNÇÃO
GET /api/health	Aberta	Healthcheck do compose. Executa <code>SELECT 1</code> no banco de propósito.
GET /api/foto/[id]	Aberta	Serve foto e vídeo do animal com cache imutável. Pública porque é ela que aparece no preview do link no WhatsApp.
GET /api/comprovante/[id]	Condicional	Só entrega o comprovante se ele estiver marcado como público; caso contrário, exige sessão de equipe.
POST /api/ia	Equipe	Recebe o histórico da conversa e devolve a resposta do modelo.

PARTE II

Arquitetura

3 Visão geral

É uma aplicação monolítica de servidor. Um container, um banco, um volume de mídia. Não há microserviço, fila, cache distribuído nem API separada do front-end — e nada disso faz falta na escala real do problema: uma ONG com noventa animais e cento e cinquenta padrinhos.



O PostgreSQL vive fora do Docker, no host da VPS. A chamada à API da Claude é tracejada porque é opcional: sem ela, tudo o mais continua funcionando.

POR QUE O BANCO FICA FORA DO DOCKER

O servidor já mantém um PostgreSQL único no host, compartilhado com outras aplicações e com backup centralizado. Subir mais um Postgres dentro do compose significaria mais uma rotina de backup para esquecer. O custo dessa escolha é a configuração de rede descrita na seção 31.

4 Stack e a razão de cada escolha

TECNOLOGIA	VERSÃO	PAPEL E MOTIVO
Next.js (App Router)	15.5	Renderização no servidor com Server Components e mutações por Server Actions. A escolha central: a maior parte das telas é leitura de banco formatada em HTML, e isso não precisa de uma API REST no meio nem de JavaScript no celular do visitante.
React	19.3	Server Components e <code>useActionState</code> , que dá estado de envio e erro de formulário sem nenhuma biblioteca de formulário.
TypeScript	5.8, <code>strict</code>	Os tipos do Prisma atravessam o sistema inteiro até o JSX. Renomear um campo do schema quebra o build, não a produção.

TECNOLOGIA	VERSÃO	PAPEL E MOTIVO
Prisma	6.19	ORM e ferramenta de migração. O <code>groupBy</code> tipado é usado à exaustão nos relatórios.
PostgreSQL	host da VPS	<code>Decimal</code> real para dinheiro, índices compostos e dois papéis de acesso com privilégios distintos.
Tailwind CSS	3.4	Paleta derivada da logo em <code>tailwind.config.ts</code> e oito classes de componente em <code>globals.css</code> . Sem CSS-in-JS: o estilo não precisa de tempo de execução.
@node-rs/argon2	2.0	Argon2id nativo para hash de senha. Parâmetros: 19 MiB de memória, 2 iterações, paralelismo 1.
Zod	3.25	Valida as variáveis de ambiente no boot e toda entrada de Server Action. Nenhum <code>FormData</code> chega ao banco sem passar por um schema.
@anthropic-ai/sdk	0.68	Assistente interno. Dependência opcional em tempo de execução.
Docker + Compose	—	Build multi-estágio, saída <code>standalone</code> do Next, rede com subnet fixa.

4.1 O que foi deixado de fora, de propósito

AUSÊNCIA	RAZÃO
Biblioteca de gráficos	São quatro formas — barras agrupadas, linha, rosca e medidor. Qualquer pacote do mercado traria dezenas de kB de JavaScript para o celular de quem só quer ver quanto entrou no mês. Os gráficos são SVG escritos à mão, em ~500 linhas (seção 28).
Biblioteca de ícones	São 23 ícones. Um pacote com mil traria mil. Ficam em <code>src/components/Icones.tsx</code> , todos com traço 1,8 — o mesmo peso do texto em negrito ao lado deles.
Fonte externa	O build não pode depender de rede, e a página pública do animal é aberta a partir de um link do WhatsApp, quase sempre em 4G ruim. A pilha de fontes é a do sistema.
Gerenciador de estado	O estado mora no banco. Os 18 componentes de cliente usam <code>useState</code> para abrir uma aba ou marcar um filtro — nada que justifique Redux, Zustand ou equivalente.
Biblioteca de formulário	<code>useActionState</code> com Server Action cobre envio, estado de carregamento e erro. Um formulário que funciona sem JavaScript é melhor do que um validado no cliente.

5 Estrutura de diretórios

```
meuaumigo/  
├─ prisma/  
│   ├─ schema.prisma      20 modelos, 20 enums, 728 linhas  
│   ├─ seed.ts            semente de demonstração (12 meses fictícios)  
│   └─ migrations/0001_init/ SQL gerado, versionado
```

└─ sql/grants.sql	privilégios do papel da aplicação
└─ scripts/	
└─ criar-usuario.ts	destranca o acesso pela linha de comando
└─ src/lib/	infraestrutura – não conhece regra de negócio
└─ env.ts	ambiente tipado; derruba o boot se faltar segredo
└─ db.ts	Prisma Client único
└─ crypto.ts	argon2 e token de sessão
└─ auth.ts	sessão, papéis, CSRF, auditoria
└─ storage.ts	mídia em disco; nome de arquivo sempre gerado aqui
└─ format.ts	dinheiro, datas, idade e rótulos dos enums
└─ paleta.ts	cores dos gráficos (fora do módulo de cliente)
└─ slug.ts	endereço público do animal
└─ src/server/	regras de negócio – só roda no servidor
└─ periodo.ts	competência mensal
└─ custos.ts	custo, cobertura e o rateio da contribuição
└─ painel.ts	agregados do painel e do impacto
└─ publico.ts	vitrine e perfil público
└─ padrinho.ts	a área do padrinho
└─ relatorios.ts	retenção, cobrança, concentração de custo
└─ transparencia.ts	prestação de contas pública
└─ gamificacao.ts	conquistas derivadas
└─ config.ts	configuração da ONG, memoizada por requisição
└─ ia/	contexto.ts + assistente.ts
└─ src/components/	10 componentes compartilhados
└─ ui.tsx	Cartao, Indicador, BarraMeta, Tabela, Aviso...
└─ Graficos.tsx	'use client' – SVG e camada de hover
└─ Icones.tsx	23 ícones
└─ FotoAnimal.tsx	foto real ou retrato gerado
└─ CartaoAnimal.tsx	o cartão da vitrine
└─ Navegacao.tsx	lateral no desktop, tira rolável no celular
└─ Compartilhar.tsx	navigator.share com queda para WhatsApp
└─ Link.tsx	fio de progresso na navegação
└─ Marca.tsx	logo
└─ BarraUsuario.tsx	identidade e sair
└─ src/app/	
└─ (publico)/	15 arquivos · 2.368 linhas
└─ (auth)/entrar/	login
└─ (padrinho)/	5 arquivos · 824 linhas
└─ (ong)/	34 arquivos · 5.899 linhas
└─ api/	4 route handlers

5.1 A regra de dependência

As camadas só olham para baixo. `app/` chama `server/`, que chama `lib/`, que fala com o banco e o disco. Nada em `lib/` importa de `server/`, e nada em `server/` importa de `app/`.

Há uma consequência prática disso que já cobrou seu preço durante o desenvolvimento e vale registrar:

ARMADILHA: CONSTATANTE EXPORTADA DE MÓDULO DE CLIENTE

A paleta dos gráficos morava dentro de `Graficos.tsx`, que é `'use client'`. Tudo que um módulo de cliente exporta vira *referência de cliente* — inclusive uma função pura como `corDaCategoria()`. Quando a página de impacto (servidora) tentou chamá-la durante o render, o Next devolveu `Attempted to call corDaCategoria() from the server` e a rota caiu com 500. A correção foi mover a paleta para `src/lib/paleta.ts`, sem diretiva: **cor é dado, não componente**. Vale a regra geral — constante compartilhada entre servidor e cliente nunca deve morar em arquivo marcado como cliente.

6 Ciclo de vida de uma requisição

6.1 Leitura — abrir uma página

Exemplo: um visitante abre `/animais/thor` a partir de um link recebido no WhatsApp.

1. O Next casa a rota com `(publico)/animais/[slug]/page.tsx`.
2. `generateMetadata()` roda primeiro e monta as tags Open Graph — título, descrição e imagem de capa. É o que o WhatsApp lê para montar o preview; sem imagem, o link vira um retângulo cinza que ninguém abre.
3. O componente servidor chama `carregarPerfilPublico(slug)` em `src/server/publico.ts`, que faz **uma** consulta com `include` aninhado para animal, fotos, apadrinhamentos ativos, diário, atualizações públicas e campanhas, e mais três contagens em paralelo.
4. `calcularCusto()` deriva a cobertura da meta a partir dos apadrinhamentos ativos já carregados — sem ida adicional ao banco.
5. O HTML é transmitido. O único JavaScript enviado ao navegador é o do botão de compartilhar. A página inteira funciona com o JavaScript desligado.

6.2 Escrita — enviar um formulário

Exemplo: a mesma pessoa confirma o apadrinhamento do Thor.

1. O `<form action={acao}>` envia o `FormData` para a Server Action `apadrinhar()`. Funciona sem JavaScript; com JavaScript, `useActionState` acrescenta o estado de envio.
2. A ação chama `conferirMesmaOrigem()` — defesa de CSRF que complementa o `SameSite` do cookie (seção 21).
3. Um schema Zod valida e normaliza tudo. Nada de `FormData` cru alcança o banco.
4. Regras de negócio: o padrinho é criado ou reaproveitado pelo e-mail, e a ação recusa o pedido se já houver apadrinhamento ativo daquela pessoa para aquele animal.
5. Uma transação grava o apadrinhamento, a primeira mensalidade pendente, o evento de diário — se for o primeiro padrinho do animal — e a notificação.
6. A auditoria é registrada fora da transação: ela não pode derrubar a operação, mas também não pode falhar em silêncio.
7. A ação devolve um estado tipado; a tela troca o formulário pela confirmação com a chave PIX.

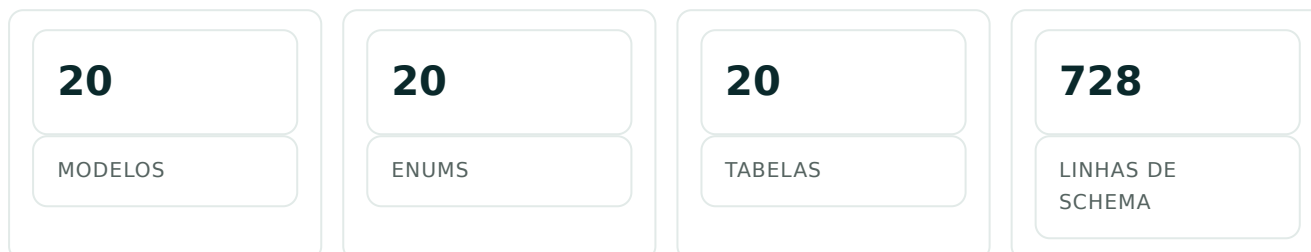
Doze arquivos `actions.ts` seguem essa mesma sequência. Quando houver dúvida sobre onde acrescentar uma regra, é essa ordem que ela deve respeitar: origem, validação, regra, transação,

auditoria, revalidação.

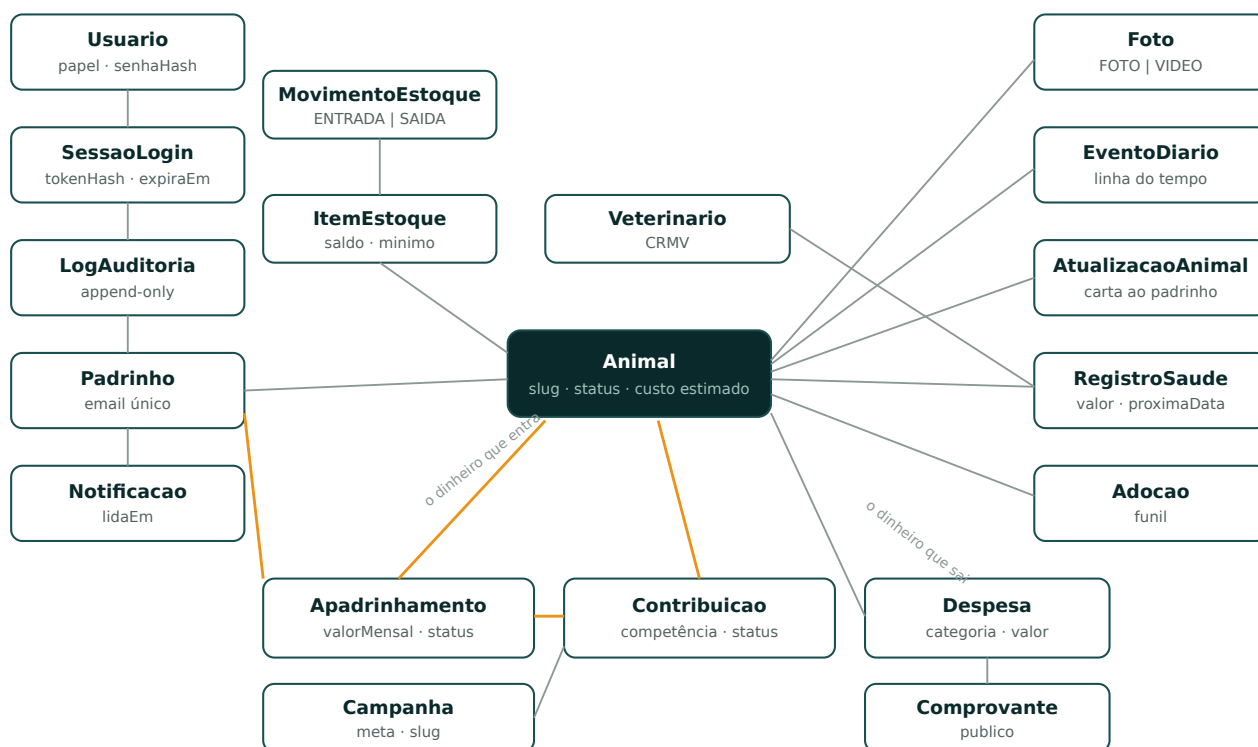
PARTE III

Modelo de dados

7 Visão do modelo



Três eixos organizam o schema — **animal**, **padrinho** e **ONG** — e o trabalho todo consiste em amarrá-los. Repare, no diagrama, que **Contribuicao** e **Despesa** apontam ambas para **Animal**: é essa dupla ligação que torna possível responder “o que os meus cem reais fizeram pelo Thor”.



Relacionamentos principais. Em laranja, o caminho do dinheiro do padrinho até o animal.

7.1 Convenções do schema

- **Identificadores:** `cuid()` em todos os modelos. Ordenável no tempo e sem revelar contagem — um id sequencial na URL conta ao visitante quantos animais a ONG já cadastrou.
- **Nomes:** modelos e campos em português, porque o domínio é falado em português pela equipe. O `@@map` aponta para tabelas no plural e em minúsculas (`animais` , `contribuicoes`).

- **Dinheiro:** sempre `Decimal @db.Decimal(10,2)` . Nunca `Float` . Peso e quantidade de estoque usam três casas.
- **Exclusão:** `deletadoEm` nos modelos com história — animal, padrinho e usuário. Apagar um animal apagaria o histórico de quem o sustentou.
- **Índices:** compostos, orientados pelas consultas reais. `@@index([competencia, status])` em contribuições e `@@index([animalId, data])` em despesas são os dois mais usados.

8 Dicionário de dados

8.1 Pessoas e acesso

Usuario • `usuarios`

Credencial de acesso. Papéis: `ADMIN` , `EQUIPE` , `VETERINARIO` e `PADRINHO` . Um padrinho com login tem registro aqui e em `Padrinho` , ligados por `usuarioId` ; a maioria dos padrinhos, porém, existe apenas em `Padrinho` , sem nunca ter criado senha.

SessaoLogin • `sessoes_login`

O banco guarda apenas o **hash SHA-256** do token; o valor em claro existe só no cookie. Um vazamento do banco não entrega sessão a ninguém. Campos de contexto — `ip` , `userAgent` — servem para investigar acesso indevido, e `revogadaEm` permite encerrar sessão sem apagar o registro.

LogAuditoria • `logs_auditoria`

Append-only de verdade: o papel da aplicação não tem `UPDATE` nem `DELETE` nesta tabela (seção 23). Guarda ação, entidade, id, um `Json` de detalhe e o IP.

Padrinho • `padrinhos`

E-mail é único e serve de chave natural: quem já apadrinha e apadrinha de novo não vira cadastro duplicado. Dois campos de consentimento merecem atenção: `aceitaNotificacao` controla o envio de novidades e `perfilPublico` autoriza o nome a aparecer no mural do animal — nenhum dos dois é presumido.

8.2 Animal e sua história

PRISMA/SCHEMA.PRISMA — MODEL ANIMAL

```

1 model Animal {
2   id    String @id @default(cuid())
3   /// Usado na URL pública: /animais/thor. É o link que vai para o WhatsApp.
4   slug  String @unique
5   nome  String
6
7   especie Especie @default(CANINA)
8   sexo    Sexo    @default(NAO_INFORMADO)
9   porte   Porte   @default(MEDIO)
10  raca     String?
11  pelagem  String?
12
13  dataNascimento DateTime?
14  /// Quando não há data de nascimento – quase sempre, em animal resgatado.
15  idadeAproximada String?
16  dataResgate    DateTime
17  localResgate    String?
18  /// 0 texto que faz alguém parar o scroll. Fica na página pública.
19  historiaResgate String?
20  personalidade   String?
21
22  castrado          Boolean @default(false)
23  vacinasEmDia      Boolean @default(false)
24  necessidadeEspecial Boolean @default(false)
25  descricaoNecessidade String?
26
27  status      StatusAnimal @default(EM_TRATAMENTO)
28  localAbrigo String?
29  pesoAtualKg Decimal?      @db.Decimal(6, 3)
30
31  /// Custo mensal estimado – é a base do "falta R$ X" na página pública.
32  /// Estimativa da ONG, revisada à mão; o gasto real vive em Despesa.
33  custoAlimentacao Decimal @default(0) @db.Decimal(10, 2)
34  custoTratamento  Decimal @default(0) @db.Decimal(10, 2)
35  custoMedicamento Decimal @default(0) @db.Decimal(10, 2)
36
37  /// Sobe para o topo da vitrine pública.
38  destaque Boolean @default(false)
39  /// Sai da vitrine sem apagar o histórico (adotado, falecido, transferido).
40  ativo Boolean @default(true)
41
42  fotoCapaId String? @unique
43
44  criadoEm    DateTime @default(now())
45  atualizadoEm DateTime @updatedAt
46  deletadoEm  DateTime?
47
48  fotoCapa      Foto? @relation("capa", fields: [fotoCapaId], references: [id],
onDelete: SetNull)
49  fotos         Foto[] @relation("galeria")
50  eventosDiario EventoDiario[]
51  registrosSaude RegistroSaude[]
52  apadrinhamentos Apadrinhamento[]
53  contribuicoes    Contribuicao[]
54  despesas         Despesa[]
55  campanhas        Campanha[]
56  adocoos          Adocao[]
57  atualizacoes      AtualizacaoAnimal[]
58  movimentos       MovimentoEstoque[]
59  notificacoes      Notificacao[]
60
61  @@index([status, ativo])

```

```

62  @@index([especie, porte])
63  @@index([nome])
64  @@map("animais")
65  }

```

Três grupos de campos merecem comentário:

CAMPO	OBSERVAÇÃO
<code>slug</code>	O endereço público (<code>/animais/thor</code>). É o link que circula no WhatsApp e no Instagram, e por isso nunca muda quando o animal é renomeado — a edição não o regenera.
<code>dataNascimento</code> + <code>idadeAproximada</code>	Animal resgatado quase nunca tem data de nascimento. Os dois campos coexistem: a exibição prefere a data e cai para a estimativa escrita à mão, marcando-a como aproximada. A ficha não pode sugerir precisão que não existe.
<code>custoAlimentacao</code> , <code>custoTratamento</code> , <code>custoMedicamento</code>	Estimativa da ONG, não gasto real. São a meta pública de custeio (seção 11).
<code>ativo</code> × <code>status</code>	<code>status</code> é a situação (em tratamento, disponível, adotado...). <code>ativo</code> tira o animal da vitrine sem apagar nada. Um animal adotado fica <code>ativo = false</code> e conserva história, despesas e padrinhos.
<code>fotoCapaId</code>	Relação um-para-um com <code>Foto</code> , separada da galeria. É a imagem que vai para o Open Graph; vídeo nunca vira capa, porque o preview do link precisa de imagem.

Foto • `fotos`

Guarda apenas o **nome** do arquivo dentro de `STORAGE_DIR/animais` — nunca caminho absoluto, porque o volume pode ser remontado em outro ponto. `tipo` distingue foto de vídeo e `ordem` define a galeria pública.

EventoDiario • `eventos_diario` e AtualizacaoAnimal • `atualizacoes_animal`

São coisas diferentes e foi deliberado separá-las. O **diário** é a biografia pública do animal: resgate, castração, primeira melhora, adoção. Alguns eventos nascem do próprio sistema (`automatico = true`) e outros a equipe escreve. A **atualização** é a carta ao padrinho: dispara notificação para quem apadrinha e só aparece no perfil público se for marcada como `publica` .

8.3 Saúde

RegistroSaude • `registros_saude`

Onze tipos, de consulta a pesagem. Três campos carregam mais peso do que parece: `valor` (que vira despesa), `pesoKg` (que atualiza o peso atual do animal quando o tipo é `PESAGEM`) e `proximaData` , que alimenta a agenda do painel e o aviso de próxima consulta na área do padrinho.

A relação `despesa Despesa?` é um-para-um opcional. É ela que garante que o mesmo atendimento apareça com o mesmo valor na ficha clínica e no financeiro, sem possibilidade de divergirem (seção 15).

8.4 O dinheiro que entra

Apadrinhamento · `apadrinhamentos`

Liga padrinho e animal com valor mensal, modalidade e status (`ATIVO` , `PAUSADO` , `CANCELADO`).

POR QUE NÃO HÁ UNICIDADE (PADRINHO, ANIMAL)

Uma pessoa pode cancelar e, meses depois, voltar a apadrinhar o mesmo animal. Duas linhas para o mesmo par são a representação correta dessa história — a restrição de “um por vez” é aplicada na regra de negócio, que recusa um novo apadrinhamento quando já existe um `ATIVO` ou `PAUSADO` .

Contribuicao · `contribuicoes`

PRISMA/SCHEMA.PRISMA — MODEL CONTRIBUICAO

```

1 model Contribuicao {
2   id          String          @id @default(cuid())
3   origem      OrigemContribuicao @default(APADRINHAMENTO)
4   apadrinhamentoId String?
5   padrinhoId   String?
6   /// Preenchido também na doação avulsa dirigida a um animal específico.
7   animalId     String?
8   campanhaId   String?
9
10  valor        Decimal         @db.Decimal(10, 2)
11  status        StatusContribuicao @default(PENDENTE)
12  metodo        MetodoPagamento?
13  /// Mês de referência, sempre no dia 1. É por ele que o financeiro fecha,
14  /// não pela data de pagamento – quem paga atrasado paga o mês certo.
15  competencia   DateTime
16  vencimento    DateTime?
17  pagoEm        DateTime?
18  /// Identificador do PIX/boleto no extrato, para conciliar.
19  referencia    String?
20  observacao    String?
21  /// Doação de quem não quer aparecer no mural.
22  anonima       Boolean         @default(false)
23  /// Nome de quem doou sem cadastro de padrinho.
24  nomeDoador    String?
25
26  criadaEm      DateTime @default(now())
27
28  apadrinhamento Apadrinhamento? @relation(fields: [apadrinhamentoId], references: [id], onDelete:
SetNull)
29  padrinho        Padrinho?        @relation(fields: [padrinhoId], references: [id], onDelete: SetNull)
30  animal          Animal?          @relation(fields: [animalId], references: [id], onDelete: SetNull)
31  campanha        Campanha?        @relation(fields: [campanhaId], references: [id], onDelete: SetNull)
32
33  @@index([competencia, status])
34  @@index([animalId, competencia])
35  @@index([padrinhoId, competencia])
36  @@index([status, vencimento])
37  @map("contribuicoes")
38 }
```

Mensalidade de apadrinhamento, doação avulsa e aporte de campanha passam todas por esta tabela. Unificá-las é o que torna o total do mês uma soma só e mantém a transparência sem buraco. O campo decisivo é `competencia` , sempre no dia 1 do mês de referência (seção 12).

8.5 O dinheiro que sai

Despesa · despesas

`animalId` é opcional: aluguel, energia e ração a granel são do abrigo inteiro. O rateio dessas despesas por animal, quando necessário, acontece no relatório — não no lançamento, que registra o fato como ele aconteceu.

Comprovante · comprovantes

O campo `publico` é a chave da transparência: a ONG escolhe o que o padrinho vê sem abrir a contabilidade inteira. Nota fiscal costuma trazer CNPJ, endereço e às vezes o nome do veterinário — publicar tudo por engano é dano que não volta.

8.6 Estoque

ItemEstoque · itens_estoque e MovimentoEstoque · movimentos_estoque

`saldo` existe no item, mas **nunca é digitado**: é mantido pelos movimentos, dentro da mesma transação que os grava. `minimo` acende o alerta de reposição no painel. Um movimento de saída pode apontar para um animal específico — ração medicamentosa, por exemplo.

8.7 Campanha, adoção, notificação e configuração

MODELO	OBSERVAÇÃO
Campanha	Tem <code>slug</code> próprio, porque também gera link compartilhável. <code>animalId</code> é opcional: há campanha do abrigo (reforma do canil) que não é de um animal. O arrecadado não é campo — é sempre somado das contribuições pagas, para não existir número que possa ficar desatualizado.
Adocao	O processo, não o resultado. <code>respostas</code> é <code>Json</code> com o que veio do formulário público (moradia, outros animais, rotina). Sete status compõem o funil.
Notificacao	Grava título, corpo e link. <code>lidaEm</code> nulo é não lida. O envio externo não existe ainda (seção 1.3).
ConfiguracaoOng	Linha única, com id fixo <code>"unica"</code> . Um <code>upsert</code> sempre acerta o alvo e nunca cria uma segunda configuração — padrão simples e à prova de duplicata.

9 Enumerações

Vinte enums no banco. Cada um tem um dicionário correspondente em `src/lib/format.ts` que traduz o valor técnico para o rótulo que a equipe usa — e, quando faz sentido, para uma cor e um emoji. O vocabulário da tela é o mesmo do grupo de WhatsApp da ONG, o que economiza um treinamento inteiro.

ENUM	VALORES
<code>PapelUsuario</code>	ADMIN · EQUIPE · VETERINARIO · PADRINHO
<code>Especie</code>	CANINA · FELINA · OUTRA
<code>Sexo</code>	MACHO · FEMEA · NAO_INFORMADO

ENUM	VALORES
Porte	PEQUENO · MEDIO · GRANDE
StatusAnimal	EM_TRATAMENTO · DISPONIVEL_ADOCAO · EM_PROCESSO_ADOCAO · ADOTADO · INDISPONIVEL · FALECIDO
TipoMidia	FOTO · VIDEO
TipoEventoDiario	RESGATE · CONSULTA · TRATAMENTO · MELHORA · PASSEIO · APADRINHAMENTO · VACINA · CASTRACAO · ADOCAO · ANIVERSARIO · OUTRO
TipoRegistroSaude	CONSULTA · VACINA · VERMIFUGO · CASTRACAO · EXAME · CIRURGIA · MEDICAMENTO · PESAGEM · ALERGIA · DOENCA · OUTRO
ModalidadeApadrinhamento	ALIMENTACAO (R\$ 30) · CUIDADOS (R\$ 50) · COMPLETO (R\$ 100) · PERSONALIZADO
StatusApadrinhamento	ATIVO · PAUSADO · CANCELADO
StatusContribuicao	PENDENTE · PAGA · ATRASADA · CANCELADA
MetodoPagamento	PIX · CARTAO · BOLETO · TRANSFERENCIA · DINHEIRO · OUTRO
OrigemContribuicao	APADRINHAMENTO · DOACAO_AVULSA · CAMPANHA
CategoriaDespesa	VETERINARIO · ALIMENTACAO · MEDICAMENTO · HIGIENE · TRANSPORTE · ESTRUTURA · OUTRO
TipoComprovante	NOTA_FISCAL · RECIBO · CUPOM · CONTRATO · OUTRO
TipoItemEstoque	RACAO · MEDICAMENTO · HIGIENE · OUTRO
TipoMovimentoEstoque	ENTRADA · SAIDA · PERDA · AJUSTE
StatusCampanha	RASCUNHO · ATIVA · CONCLUIDA · ENCERRADA
StatusAdocao	INTERESSE · ENTREVISTA · VISITA · APROVADA · CONCLUIDA · RECUSADA · DESISTENCIA
TipoNotificacao	ATUALIZACAO_ANIMAL · APADRINHAMENTO_CONFIRMADO · PAGAMENTO_PENDENTE · ANIVERSARIO · CONQUISTA · CAMPANHA · ADOCAO · OUTRO

10 Migrations e privilégios

A migração inicial (`0001_init`, 608 linhas de SQL) é versionada e aplicada pelo entrypoint com o papel **dono**. Logo depois, o mesmo entrypoint reaplica os privilégios da aplicação:

PRISMA/SQL/GRANTS.SQL

```
1 -- =====
2 -- GRANTS do papel da aplicação.
3 --
4 -- Roda a cada deploy, com o papel DONO, logo depois das migrations.
5 -- Idempotente de propósito: tabela criada por migration futura já nasce
6 -- acessível à app, e se alguém afrouxar a muralha à mão, o próximo
7 -- deploy a reafirma.
8 --
9 -- `meuaumigo_app` NUNCA recebe CREATE/ALTER/DROP: a aplicação não pode
10 -- alterar o próprio schema.
11 -- =====
12
13 GRANT USAGE ON SCHEMA public TO meuaumigo_app;
14
15 GRANT SELECT, INSERT, UPDATE, DELETE
16   ON ALL TABLES IN SCHEMA public TO meuaumigo_app;
17
18 GRANT USAGE, SELECT ON ALL SEQUENCES IN SCHEMA public TO meuaumigo_app;
19
20 ALTER DEFAULT PRIVILEGES IN SCHEMA public
21   GRANT SELECT, INSERT, UPDATE, DELETE ON TABLES TO meuaumigo_app;
22
23 ALTER DEFAULT PRIVILEGES IN SCHEMA public
24   GRANT USAGE, SELECT ON SEQUENCES TO meuaumigo_app;
25
26 -- O histórico de migrations é do dono.
27 REVOKE ALL ON TABLE "_prisma_migrations" FROM meuaumigo_app;
28
29 -- Auditoria é append-only, nem para a app. Sistema que administra
30 -- dinheiro de terceiro precisa de log que ele próprio não reescreva.
31 REVOKE UPDATE, DELETE ON TABLE "logs_auditoria" FROM meuaumigo_app;
```

O arquivo é idempotente de propósito. Tabela criada por migração futura já nasce acessível à aplicação por causa do `ALTER DEFAULT PRIVILEGES`, e se alguém afrouxar a muralha à mão, o próximo deploy a reafirma. As duas últimas instruções são as mais importantes do arquivo: tiram o histórico de migrações do alcance da aplicação e tornam a auditoria imutável para ela.

PARTE IV

Regras de negócio

Esta é a parte do documento que mais vale ler antes de alterar código. Quase toda regra aqui existe para resolver um conflito real entre duas coisas desejáveis, e a escolha está registrada junto com o motivo.

11 Custo estimado × gasto real

O sistema mantém **dois números diferentes** para o custo de um animal, e confundi-los é o erro mais fácil de cometer aqui.

	CUSTO ESTIMADO	GASTO REAL
Onde mora	Campos do próprio <code>Animal</code>	Tabela <code>Despesa</code>
Quem define	A equipe, revisando à mão	Os lançamentos do dia a dia
Para que serve	Meta pública de custeio, valor de referência do apadrinhamento	Prestação de contas, relatórios, rateio
Estabilidade	Muda raramente	Muda todo dia

DECISÃO

A barra de meta da página pública usa o **estimado**, não o gasto do mês. O apadrinhamento é recorrente e precisa de um alvo estável: um mês com cirurgia não pode fazer a meta do Thor saltar de R\$ 500 para R\$ 3.000 na vitrine e desanimar quem ia contribuir com R\$ 50. Para despesa excepcional existe **campanha** (seção 18).

SRC/SERVER/CUSTOS.TS — A COBERTURA DA META

```
1 export function calcularCusto(animal: AnimalComCusto): CustoAnimal {
2   const alimentacao = paraNumero(animal.custoAlimentacao);
3   const tratamento = paraNumero(animal.custoTratamento);
4   const medicamento = paraNumero(animal.custoMedicamento);
5   const total = alimentacao + tratamento + medicamento;
6
7   const ativos = (animal.apadrinhamentos ?? []).filter((a) => a.status === 'ATIVO');
8   const apoiado = ativos.reduce((soma, a) => soma + paraNumero(a.valorMensal), 0);
9
10  return {
11    alimentacao,
12    tratamento,
13    medicamento,
14    total,
15    apoiado,
16    falta: Math.max(0, total - apoiado),
17    percentual: total > 0 ? Math.min(100, (apoiado / total) * 100) : apoiado > 0 ? 100 : 0,
18    padrinhos: ativos.length,
19  };
20 }
```

Três detalhes dessa função valem atenção. O percentual é travado em 100 — apoio acima da meta não vira 140% na barra. Um animal sem custo cadastrado mas com padrinho conta como 100%, não como divisão por zero. E a função não consulta o banco: recebe os apadrinhamentos já carregados pela página, o que permite calcular a cobertura de noventa animais na vitrine sem noventa consultas.

12 Competência: como o mês fecha

Uma mensalidade pertence ao mês que ela *cobre*, não ao dia em que o dinheiro entrou. Quem paga setembro no dia 3 de outubro pagou setembro.

- **Contribuição** conta pelo campo `competencia`, sempre no dia 1 do mês de referência.
- **Despesa** conta pela `data` do lançamento, porque o gasto aconteceu naquele dia.

Essa assimetria é intencional e está anotada no rodapé das telas de painel e financeiro, para que ninguém tente conciliar os dois números com a régua errada. Todo o cálculo de mês passa por `src/server/periodo.ts`, que é o único lugar do sistema autorizado a decidir onde um mês começa.

FUSO HORÁRIO

`inicioDoMes()` usa o fuso do processo. O `docker-compose.yml` fixa `TZ=America/Sao_Paulo` porque a imagem base roda em UTC — e, sem isso, no dia 1º toda competência calculada entre 21h e 23h59 cairia no mês seguinte. Mexer nessa variável quebra o fechamento financeiro de forma silenciosa.

13 O rateio da contribuição

É o cálculo que sustenta a promessa do produto: mostrar ao padrinho em que o dinheiro dele virou cuidado. Recebe o animal, o valor pago e a competência, e devolve a divisão por categoria.

SRC/SERVER/CUSTOS.TS — RATEARCONTRIBUICAO()

```
1 export async function ratearContribuicao(
2   animalId: string,
3   valorPago: number,
4   competencia: Date,
5 ): Promise<{ categoria: string; rotulo: string; valor: number }[]> {
6   const fim = new Date(competencia.getFullYear(), competencia.getMonth() + 1, 1);
7
8   const despesas = await db.despesa.groupBy({
9     by: ['categoria'],
10    where: { animalId, data: { gte: competencia, lt: fim } },
11    _sum: { valor: true },
12  });
13
14  let pesos = despesas.map((d) => ({
15    categoria: d.categoria as string,
16    peso: paraNumero(d._sum.valor),
17  }));
18  let totalPesos = pesos.reduce((s, p) => s + p.peso, 0);
19
20  if (totalPesos <= 0) {
21    const animal = await db.animal.findUnique({
22      where: { id: animalId },
23      select: { custoAlimentacao: true, custoTratamento: true, custoMedicamento: true },
24    });
25    pesos = [
26      { categoria: 'ALIMENTACAO', peso: paraNumero(animal?.custoAlimentacao) },
27      { categoria: 'VETERINARIO', peso: paraNumero(animal?.custoTratamento) },
28      { categoria: 'MEDICAMENTO', peso: paraNumero(animal?.custoMedicamento) },
29    ];
30    totalPesos = pesos.reduce((s, p) => s + p.peso, 0);
31  }
32
33  if (totalPesos <= 0) return [];
34
35  const rotulos: Record<string, string> = {
36    VETERINARIO: 'Veterinário',
37    ALIMENTACAO: 'Alimentação',
38    MEDICAMENTO: 'Medicamentos',
39    HIGIENE: 'Higiene',
40    TRANSPORTE: 'Transporte',
41    ESTRUTURA: 'Estrutura',
42    OUTRO: 'Outros',
43  };
44
45  const linhas = pesos
46    .filter((p) => p.peso > 0)
47    .map((p) => ({
48      categoria: p.categoria,
49      rotulo: rotulos[p.categoria] ?? p.categoria,
50      valor: Math.round((valorPago * p.peso * 100) / totalPesos) / 100,
51    }))
52    .sort((a, b) => b.valor - a.valor);
53
54  // 0 arredondamento de centavos sobra ou falta uns trocados; joga a
55  // diferença na maior linha para que a soma bata com o que foi pago.
56  const somado = linhas.reduce((s, l) => s + l.valor, 0);
57  const resto = Math.round((valorPago - somado) * 100) / 100;
58  if (linhas.length > 0 && resto !== 0) linhas[0].valor = Math.round((linhas[0].valor + resto) * 100) / 100;
59
60  return linhas;
61 }
```

O algoritmo tem três camadas de cuidado:

1. **Base real primeiro.** Agrupa as despesas daquele animal naquele mês por categoria e usa essa proporção. É o retrato honesto.
2. **Queda para a estimativa.** Quando o animal não teve despesa lançada no mês — o que acontece e é normal —, usa a proporção do custo estimado. A tela deixa claro que, nesse caso, é previsão.
3. **O total sempre bate.** O arredondamento de centavos sobra ou falta alguns trocados; a diferença é jogada na maior linha. A pessoa nunca vê uma soma diferente da que saiu da conta dela — e é esse detalhe que impede uma conversa constrangedora no WhatsApp da ONG.

14 Ciclo do apadrinhamento

14.1 Nascimento

É o único fluxo de escrita aberto a quem não tem login, e por isso é o mais conservador do sistema. O padrinho é identificado pelo e-mail com `upsert`; a atualização só preenche campos vazios, para que um cadastro antigo e mais completo não seja esvaziado por um formulário preenchido às pressas. Em seguida, uma transação:

ACTIONS.TS — A TRANSAÇÃO DO APADRINHAMENTO

```
1  await db.$transaction(async (tx) => {
2    const apadrinhamento = await tx.apadrinhamento.create({
3      data: {
4        padrinhoId: padrinho.id,
5        animalId: animal.id,
6        modalidade: entrada.modalidade,
7        valorMensal,
8        exibirNoPerfil: entrada.exibirNoPerfil,
9      },
10   });
11
12   // A primeira mensalidade já nasce, pendente. Sem ela o apadrinhamento
13   // fica invisível no financeiro até alguém lembrar de lançar.
14   await tx.contribuicao.create({
15     data: {
16       origem: 'APADRINHAMENTO',
17       apadrinhamentoId: apadrinhamento.id,
18       padrinhoId: padrinho.id,
19       animalId: animal.id,
20       valor: valorMensal,
21       status: 'PENDENTE',
22       competencia,
23       vencimento: new Date(competencia.getFullYear(), competencia.getMonth(), 10),
24     },
25   });
26
27   if (primeiroPadrinho) {
28     await tx.eventoDiario.create({
29       data: {
30         animalId: animal.id,
31         tipo: 'APADRINHAMENTO',
32         data: new Date(),
33         titulo: `${animal.nome} ganhou seu primeiro padrinho!`,
34         automatico: true,
35       },
36     });
37   }
38
39   await tx.notificacao.create({
40     data: {
41       padrinhoId: padrinho.id,
42       animalId: animal.id,
43       tipo: 'APADRINHAMENTO_CONFIRMADO',
44       titulo: 'Seu apadrinhamento foi registrado ❤️',
45       corpo: `Obrigado por cuidar do ${animal.nome}. Assim que confirmarmos o primeiro pagamento,
46       você começa a receber as novidades dele.`,
47       link: `/animais/${animal.slug}`,
48     },
49   });
50 }
```

A primeira mensalidade já nasce junto, pendente. Sem isso, o apadrinhamento ficaria invisível no financeiro até alguém lembrar de lançá-lo — e ninguém lembra.

14.2 Cobrança mensal

Não há agendador na VPS. Quem dispara a cobrança é a equipe, por um botão em `/financeiro`. Um botão é clicado duas vezes, então a ação foi escrita para ser idempotente:

FINANCEIRO/ACTIONS.TS — GERAÇÃO IDEMPOTENTE

```

1 export async function gerarMensalidades(): Promise<EstadoFinanceiro> {
2   await conferirMesmaOrigem();
3   const usuario = await exigirAdmin();
4
5   const competencia = inicioDoMes();
6   const proximo = new Date(competencia.getFullYear(), competencia.getMonth() + 1, 1);
7
8   const ativos = await db.apadrinhamento.findMany({
9     where: { status: 'ATIVO' },
10    select: { id: true, padrinhoId: true, animalId: true, valorMensal: true, diaVencimento: true },
11  });
12
13  const jaLancados = new Set(
14    (
15      await db.contribuicao.findMany({
16        where: {
17          origem: 'APADRINHAMENTO',
18          competencia: { gte: competencia, lt: proximo },
19          apadrinhamentoId: { in: ativos.map((a) => a.id) },
20        },
21        select: { apadrinhamentoId: true },
22      })
23    ).map((c) => c.apadrinhamentoId),
24  );
25
26  const novos = ativos.filter((a) => !jaLancados.has(a.id));
27  if (novos.length === 0) return { ok: 'As mensalidades deste mês já estavam geradas.' };
28
29  await db.contribuicao.createMany({
30    data: novos.map((a) => ({
31      origem: 'APADRINHAMENTO' as const,
32      apadrinhamentoId: a.id,
33      padrinhoId: a.padrinhoId,
34      animalId: a.animalId,
35      valor: a.valorMensal,
36      status: 'PENDENTE' as const,
37      competencia,
38      vencimento: new Date(competencia.getFullYear(), competencia.getMonth(), a.diaVencimento),
39    })),
40  });
41
42  await registrarAuditoria({
43    usuarioId: usuario.id,
44    acao: 'financeiro.mensalidades_geradas',
45    entidade: 'Contribuicao',
46    detalhe: { competencia: competencia.toISOString(), quantidade: novos.length },
47  });
48
49  revalidatePath('/financeiro');
50  return { ok: `${novos.length} mensalidades geradas.` };
51 }

```

A idempotência vem do conjunto `jaLancados`: apadrinhamento que já tem lançamento naquela competência é simplesmente pulado, e a ação informa quantas mensalidades foram criadas — ou que já estavam todas geradas.

14.3 Pausa e cancelamento

A interface trata os dois de formas deliberadamente assimétricas. Pausar é um clique e volta com outro. Cancelar abre um campo de motivo obrigatório antes de confirmar, porque encerra o vínculo e

tem consequência financeira:

- o `fim` é gravado e alimenta o gráfico de cancelamentos do painel — reativar limpa o campo, senão o mês antigo continuaria contando a saída;
- toda mensalidade `PENDENTE` ou `ATRASADA` daquele apadrinhamento vira `CANCELADA`. Cobrar mensalidade futura de quem cancelou é cobrança indevida;
- o que já foi pago permanece intocado — é história.

15 Saúde amarrada ao financeiro

Quando a equipe registra um atendimento com valor, ele vira despesa daquele animal **na mesma transação**, com o vínculo guardado em `Despesa.registroSaudeId` — não uma cópia solta.

SAUDE/ACTIONS.TS — REGISTRO CLÍNICO E SEUS EFEITOS

```

1  await db.$transaction(async (tx) => {
2    const registro = await tx.registroSaude.create({
3      data: {
4        animalId: e.animalId,
5        tipo: e.tipo,
6        data: new Date(e.data),
7        titulo: e.titulo,
8        descricao: e.descricao,
9        valor: e.valor ?? null,
10       pesoKg: e.pesoKg ?? null,
11       proximaData: e.proximaData ? new Date(e.proximaData) : null,
12       medicamento: e.medicamento,
13       dosagem: e.dosagem,
14       veterinarioId: e.veterinarioId || null,
15     },
16   });
17
18   if (e.valor && e.valor > 0 && e.lancarDespesa) {
19     await tx.despesa.create({
20       data: {
21         categoria:
22           e.tipo === 'MEDICAMENTO' ? 'MEDICAMENTO' : e.tipo === 'VACINA' ? 'MEDICAMENTO' :
23           'VETERINARIO',
24         animalId: e.animalId,
25         descricao: `${e.titulo} - ${animal.nome}`,
26         valor: e.valor,
27         data: new Date(e.data),
28         registroSaudeId: registro.id,
29       },
30     });
31   }
32
33   // A pesagem atualiza o peso atual do animal: a ficha nunca fica
34   // mostrando o peso de quando ele chegou.
35   if (e.tipo === 'PESAGEM' && e.pesoKg) {
36     await tx.animal.update({ where: { id: e.animalId }, data: { pesoAtualKg: e.pesoKg } });
37   }
38
39   if (e.tipo === 'CASTRACAO') {
40     await tx.animal.update({ where: { id: e.animalId }, data: { castrado: true } });
41   }
42
43   // Castração e vacina entram no diário público: são marcos que o
44   // padrinho quer ver. Consulta e exame ficam só na ficha clínica.
45   if (e.tipo === 'CASTRACAO' || e.tipo === 'VACINA') {
46     await tx.eventoDiario.create({
47       data: {
48         animalId: e.animalId,
49         tipo: e.tipo,
50         data: new Date(e.data),
51         titulo: e.tipo === 'CASTRACAO' ? `${animal.nome} foi castrado.` : e.titulo,
52         automatico: true,
53         autorId: usuario.id,
54       },
55     });
56   }
57 });

```

Essa transação faz quatro coisas além de gravar o registro:

- cria a despesa, classificando-a como medicamento ou veterinário conforme o tipo;

- uma **pesagem** atualiza o peso atual do animal, para que a ficha não fique mostrando o peso de quando ele chegou;
- uma **castração** marca o animal como castrado;
- castração e vacina entram no diário público, porque são marcos que o padrinho quer ver. Consulta e exame ficam só na ficha clínica — o diário é biografia, não prontuário.

16 Estoque: saldo é consequência

O saldo de um item nunca é digitado. Ele é calculado a partir do tipo de movimento e aplicado com `increment` dentro da transação:

ALIMENTACAO/ACTIONS.TS — O DELTA DO MOVIMENTO

```
1  const saldoAtual = Number(item.saldo);
2  const delta =
3    e.tipo === 'ENTRADA' ? e.quantidade : e.tipo === 'AJUSTE' ? e.quantidade - saldoAtual : -
    e.quantidade;
4
5  if (saldoAtual + delta < 0) {
6    return {
7      erro: `Saldo insuficiente: há ${saldoAtual} ${item.unidade} de ${item.nome} em estoque.`,
8    };
9  }
10
11  await db.$transaction(async (tx) => {
12    await tx.movimentoEstoque.create({
13      data: {
14        itemId: e.itemId,
15        tipo: e.tipo,
16        quantidade: e.tipo === 'AJUSTE' ? Math.abs(delta) : e.quantidade,
17        valor: e.valor ?? null,
18        fornecedor: e.fornecedor,
19        animalId: e.animalId || null,
20        data: new Date(e.data),
21        observacao: e.observacao,
22      },
23    });
24
25    await tx.itemEstoque.update({
26      where: { id: e.itemId },
27      data: { saldo: { increment: delta } },
28    });
29
30    if (e.tipo === 'ENTRADA' && e.valor && e.valor > 0 && e.lancarDespesa) {
31      await tx.despesa.create({
32        data: {
33          categoria:
34            item.tipo === 'RACAO' ? 'ALIMENTACAO' : item.tipo === 'MEDICAMENTO' ? 'MEDICAMENTO' :
35            'HIGIENE',
36          animalId: e.animalId || null,
37          descricao: `${item.nome} - ${e.quantidade} ${item.unidade}`,
38          valor: e.valor,
39          data: new Date(e.data),
40          fornecedor: e.fornecedor,
41        },
42      });
43    });
```

Um **ajuste de inventário** informa o saldo contado, e o sistema deriva a diferença — é assim que a contagem física entra sem quebrar a rastreabilidade. Saída que deixaria o saldo negativo é recusada com uma mensagem que diz quanto há em estoque, em vez de um erro genérico.

Uma **entrada com valor** também vira despesa de alimentação (ou medicamento, ou higiene, conforme o tipo do item), para que a compra de ração apareça no financeiro sem ninguém lançar duas vezes.

A previsão de dias exibida na tela de alimentação é o saldo total de ração dividido pelo consumo médio diário dos últimos 30 dias — calculado a partir dos movimentos de saída, não de uma média digitada.

17 Funil de adoção

Sete status, mas a interface só oferece o **próximo passo** possível:



cada avanço muda o status do animal na vitrine pública; a recusa só o devolve se nenhum outro processo estiver correndo

O funil. Status intermediários colocam o animal em EM_PROCESSO_ADOCAO; a conclusão o marca como ADOTADO e o retira da vitrine.

ADOCOS/ACTIONS.TS — O FUNIL E O STATUS DO ANIMAL

```

1  await db.$transaction(async (tx) => {
2    await tx.adocao.update({
3      where: { id: e.id },
4      data: {
5        status: e.status,
6        observacoes: e.observacoes,
7        concluidaEm: e.status === 'CONCLUIDA' ? new Date() : null,
8      },
9    });
10
11   if (e.status === 'CONCLUIDA') {
12     await tx.animal.update({
13       where: { id: adocao.animalId },
14       data: { status: 'ADOTADO', ativo: false },
15     });
16     await tx.eventoDiario.create({
17       data: {
18         animalId: adocao.animalId,
19         tipo: 'ADOCADO',
20         data: new Date(),
21         titulo: `${adocao.animal.nome} foi adotado! 🏠`,
22         descricao: `${adocao.animal.nome} encontrou uma família.`,
23         automatico: true,
24         autorId: usuario.id,
25       },
26     });
27   } else if (['ENTREVISTA', 'VISITA', 'APROVADA'].includes(e.status)) {
28     await tx.animal.update({
29       where: { id: adocao.animalId },
30       data: { status: 'EM_PROCESSO_ADOCAO' },
31     });
32   } else if (['RECUSADA', 'DESISTENCIA'].includes(e.status)) {
33     // Só volta para a vitrine se nenhum outro processo estiver correndo.
34     const outros = await tx.adocao.count({
35       where: {
36         animalId: adocao.animalId,
37         id: { not: e.id },
38         status: { in: ['INTERESSE', 'ENTREVISTA', 'VISITA', 'APROVADA'] },
39       },
40     });
41     if (outros === 0) {
42       await tx.animal.update({
43         where: { id: adocao.animalId },
44         data: { status: 'DISPONIVEL_ADOCAO' },
45       });
46     }
47   }
48 });

```

POR QUE NÃO UM SELETOR COM OS SETE STATUS

Um `<select>` deixaria “concluída” a um clique de distância de “interesse”. Concluir uma adoção marca o animal como adotado, o desativa e fecha a página pública dele. A interface oferece apenas as transições legítimas a partir do estado atual — a regra do funil vive na tela e na ação.

Repare na contagem de `outros` antes de devolver o animal à vitrine: se duas pessoas estavam interessadas e uma desistiu, o animal continua em processo por causa da outra.

18 Campanhas e reconhecimento

18.1 Campanha

O valor arrecadado de uma campanha **não é um campo**. É sempre a soma das contribuições com status `PAGA` ligadas a ela. Guardar um total seria criar um número que pode ficar desatualizado — e o número errado aqui aparece em letra garrafal numa página pública.

A tela da ONG mostra separadamente o que já foi confirmado e o que está aguardando conferência, para que a equipe saiba que a barra vai subir.

18.2 Gamificação

Reconhecimento, não competição: não há ranking, pontuação nem comparação entre padrinhos. O que existe é lembrar à pessoa há quanto tempo ela sustenta aquele animal, porque é isso que segura o apadrinhamento no décimo terceiro mês.

`SRC/SERVER/GAMIFICACAO.TS`

```
1 export function calcularConquistas(d: DadosConquista): Conquista[] {
2   const marco = (meses: number, titulo: string, emoji: string, descricao: string): Conquista => ({
3     codigo: `tempo-${meses}`,
4     titulo,
5     emoji,
6     descricao,
7     conquistada: d.mesesMaisAntigo >= meses,
8     progresso: Math.min(100, (d.mesesMaisAntigo / meses) * 100),
9   });
10
11   return [
12     {
13       codigo: 'primeiro',
14       titulo: 'Primeiro AUmigo',
15       emoji: '🐾',
16       descricao: 'Você apadrinhou seu primeiro animal.',
17       conquistada: d.animaisApoiados >= 1,
18       progresso: d.animaisApoiados >= 1 ? 100 : 0,
19     },
20     marco(6, '6 meses de apadrinhamento', '❤️', 'Meio ano cuidando da mesma história.'),
21     marco(12, '1 ano de amizade', '🏆', 'Um ano inteiro ao lado do mesmo AUmigo.'),
22     {
23       codigo: 'varios',
24       titulo: 'AUmigo de vários animais',
25       emoji: '🌟',
26       descricao: 'Você apadrinha dois ou mais animais ao mesmo tempo.',
27       conquistada: d.animaisAtivos >= 2,
28       progresso: Math.min(100, (d.animaisAtivos / 2) * 100),
29     },
30     {
31       codigo: 'campanha',
32       titulo: 'Presente na emergência',
33       emoji: '🚑',
34       descricao: 'Você ajudou numa campanha de emergência.',
35       conquistada: d.apoiouCampanha,
36       progresso: d.apoiouCampanha ? 100 : 0,
37     },
38   ];
39 }
```

DECISÃO

Conquistas são **derivadas a cada render**, nunca gravadas. Conquista armazenada é conquista que fica errada quando o dado muda — um apadrinhamento corrigido para trás faria a medalha de “1 ano” mentir para sempre. O custo é recalcular; o benefício é nunca ter de reconciliar.

PARTE V

Segurança

O sistema administra dinheiro de terceiros e dados pessoais de cento e cinquenta pessoas que confiaram na ONG. As defesas abaixo estão ordenadas da mais estrutural para a mais pontual.

19 Dois papéis no banco

A aplicação **não pode alterar o próprio schema**. Existem duas credenciais para o mesmo banco:

PAPEL	QUEM USA	PRIVILÉGIOS
<code>meuaumigo_app</code>	A aplicação em execução, via <code>DATABASE_URL</code>	<code>SELECT</code> , <code>INSERT</code> , <code>UPDATE</code> , <code>DELETE</code> . Nenhum <code>CREATE</code> , <code>ALTER</code> ou <code>DROP</code> . Sem <code>UPDATE</code> / <code>DELETE</code> na auditoria e sem acesso ao histórico de migrações.
<code>meuaumigo_owner</code>	Apenas o entypoint, via <code>DATABASE_URL_OWNER</code>	Dono do schema. Usado só por <code>prisma migrate deploy</code> e pela aplicação dos GRANTS, no boot.

A separação transforma uma classe inteira de incidentes em erro barrado pelo banco. Um bug de código, uma injeção que escapasse da camada do ORM ou uma dependência comprometida não conseguem apagar tabela nem reescrever o log de auditoria: o papel que está conectado simplesmente não tem essa permissão.

20 Sessão e senha

20.1 Senha

Argon2id via `@node-rs/argon2` , com 19 MiB de memória, 2 iterações e paralelismo 1 — os parâmetros recomendados pelo OWASP para uso interativo. A senha em claro nunca é gravada nem registrada em log.

O login responde a mesma mensagem — “*E-mail ou senha incorretos*” — para usuário inexistente, senha errada e conta desativada. E, no caminho do usuário inexistente, ainda executa uma verificação contra um hash descartável: sem isso, a diferença de tempo de resposta viraria um oráculo de “este e-mail existe no sistema”.

20.2 Token de sessão

SRC/LIB/AUTH.TS — LEITURA MEMOIZADA DA SESSÃO

```
1 export const lerSessao = cache(async (): Promise<UsuarioSessao | null> => {
2   const token = (await cookies()).get(COOKIE)?.value;
3   if (!token) return null;
4
5   const sessao = await db.sessaoLogin.findUnique({
6     where: { tokenHash: hashToken(token) },
7     include: { usuario: { include: { padrinho: { select: { id: true } } } } },
8   });
9
10  if (!sessao || sessao.revogadaEm || sessao.expiraEm < new Date()) return null;
11  if (!sessao.usuario.ativo || sessao.usuario.deletadoEm) return null;
12
13  return {
14    id: sessao.usuario.id,
15    nome: sessao.usuario.nome,
16    email: sessao.usuario.email,
17    papel: sessao.usuario.papel,
18    padrinhoId: sessao.usuario.padrinho?.id ?? null,
19  };
20 });
```

- O token é **32 bytes de aleatoriedade real** em hexadecimal.
- O banco guarda apenas o SHA-256 dele. Um dump não dá sessão a ninguém. SHA-256 basta aqui porque o token já é aleatório e longo — não há o que adivinhar por força bruta, e o custo de um Argon2 a cada requisição seria desperdício.
- O cookie é `httpOnly`, `SameSite=Lax` e `Secure` quando há TLS. Atrás do Cloudflare Tunnel o TLS termina no proxy, então a decisão também aceita o cabeçalho `x-forwarded-proto`.
- A leitura é envolvida em `cache()` do React: vários usos no mesmo render batem no banco uma vez só.
- Cada leitura reconfere expiração, revogação, `ativo` e `deletadoEm`. Desativar um usuário revoga as sessões dele na mesma ação — sem isso, a pessoa continuaria dentro até o cookie expirar, o que pode ser meio dia inteiro.

21 Defesa de CSRF

SRC/LIB/AUTH.TS — CONFERIRMESMAORIGEM()

```

1 export async function conferirMesmaOrigem(): Promise<void> {
2   const h = await headers();
3   const origem = h.get('origin');
4   if (!origem) return; // navegação same-origin não envia Origin
5
6   const hostDaRequisicao = h.get('x-forwarded-host') ?? h.get('host');
7   const permitidas = new Set([new URL(env.APP_URL).origin]);
8   if (hostDaRequisicao) {
9     const protocolo = h.get('x-forwarded-proto') ?? (ehProducao ? 'https' : 'http');
10    permitidas.add(`${protocolo}://${hostDaRequisicao}`);
11  }
12
13  if (!permitidas.has(origem)) {
14    console.warn(`[auth] origem recusada: ${origem} (esperado: ${[...permitidas].join(' ou ')}`);
15    throw new Error('Origem não permitida.');
```

Toda Server Action começa por essa checagem. Ela aceita duas origens: a configurada em `APP_URL` e a da própria requisição, reconstruída a partir do host encaminhado pelo proxy. A segunda existe porque o app fica atrás de um túnel — se o domínio mudar e alguém esquecer de ajustar a variável, o login pararia de funcionar com um erro que não explica nada.

Isso não afrouxa a proteção: num POST vindo de outro site, o `Origin` é o do site atacante e continua diferente do host da requisição.

22 Autorização por papel

Quatro funções em `src/lib/auth.ts` são o único portão de entrada, usadas tanto nos layouts quanto no início de cada Server Action:

FUNÇÃO	COMPORTAMENTO
<code>exigirUsuario()</code>	Sessão válida ou redireciona para <code>/entrar</code> .
<code>exigirEquipe()</code>	Papel em ADMIN, EQUIPE ou VETERINARIO. Um padrinho autenticado que digite <code>/painel</code> não recebe “acesso negado”: é levado para <code>/meus-aumigos</code> , que é onde ele quer estar.
<code>exigirAdmin()</code>	Financeiro, configurações e geração de cobrança. Lança erro em vez de redirecionar — quem chegou aqui sem ser admin está tentando algo que a interface não oferece.
<code>exigirPadrinho()</code>	Exige vínculo com um registro de <code>Padrinho</code> ; o <code>padrinhoId</code> vem da sessão, nunca da URL.

A VERIFICAÇÃO É NO SERVIDOR, SEMPRE

O menu esconde `/financeiro` de quem não é admin, mas esconder não é proteger. A página chama `exigirAdmin()` antes de qualquer consulta, e a Server Action que gera cobranças chama de novo. Esconder é cortesia com o usuário; barrar é a segurança.

Na área do padrinho, todas as consultas são filtradas pelo `padrinhoId` da sessão. A rota `/meus-aumigos/[slug]` procura o apadrinhamento por `padrinhoId` e slug do animal — trocar o slug na barra de endereço devolve 404, não os dados de outra pessoa.

23 Auditoria

Toda mudança de dinheiro, de apadrinhamento e de status de animal é registrada: quem fez, o quê, em qual entidade, com que detalhe e de qual IP. Quando um padrinho perguntar por que a mensalidade dele mudou, a resposta existe.

Duas escolhas sustentam a confiabilidade desse log:

- **É append-only no banco**, não por convenção de código. O `REVOKE UPDATE, DELETE` do `grants.sql` significa que nem a aplicação consegue reescrever uma linha. Um incidente precisa de log que o próprio sistema comprometido não consiga alterar.
- **Falha de auditoria não derruba a operação** — mas também não passa em silêncio: o erro vai para o log do container. A operação do usuário é mais importante do que o registro dela, e um registro que falha sem avisar é pior do que nenhum.

24 Mídia, comprovantes e dados pessoais

24.1 Arquivos

O nome do arquivo é **sempre gerado pelo sistema** (`timestamp` em base 36 + 8 bytes aleatórios + extensão derivada do tipo MIME). O nome que veio do navegador nunca toca o sistema de arquivos. Na leitura, o caminho passa por `path.basename()`, o que fecha a porta de um `../../etc/passwd` que viesse do banco.

Os tipos aceitos são uma lista de permissão explícita — JPEG, PNG, WEBP, AVIF, MP4, WEBM, MOV para mídia; PDF e imagens para comprovante — e o limite é 25 MB por arquivo.

24.2 Foto pública, comprovante condicional

As duas rotas de arquivo têm regras de acesso deliberadamente diferentes:

- `/api/foto/[id]` é **pública**. É ela que aparece no preview do link compartilhado no WhatsApp, e um preview que exige login não aparece. Responde com cache imutável, porque o arquivo nunca muda depois de escrito.
- `/api/comprovante/[id]` só entrega o documento se ele estiver marcado como `publico`; caso contrário, exige sessão de equipe — e responde **404**, não 403, para não confirmar a existência do documento a quem não deveria vê-lo.

24.3 Dados pessoais e a IA

O contexto enviado ao modelo é montado no servidor, tem tamanho conhecido e contém apenas **agregados e dados de animais**. Nome, e-mail, telefone e documento de padrinho não saem daqui (seção 29).

24.4 Superfície pública de escrita

Três ações aceitam entrada de quem não está autenticado. Vale conhecê-las, porque são o alvo natural de abuso:

AÇÃO	CONTENÇÃO
<code>apadrinhar()</code>	Valida com Zod, confere origem, recusa duplicidade de apadrinhamento ativo e cria tudo como <code>PENDENTE</code> . Nenhum registro criado por essa via concede acesso ou aparece como dinheiro recebido.
<code>doar()</code>	Mesma contenção. Valor mínimo de R\$ 5 e máximo de R\$ 100.000 impedem que a barra de uma campanha seja poluída com um número absurdo.
<code>manifestarInteresse()</code>	Abre processo com status <code>INTERESSE</code> e recusa um segundo pedido em aberto do mesmo e-mail para o mesmo animal.

LIMITAÇÃO CONHECIDA

Não há *rate limit* nem CAPTCHA nessas três rotas. Na escala atual — uma ONG divulgando para a própria rede — isso não foi um problema, e o dano de um abuso é limitado a registros pendentes que a equipe descarta. Se o sistema ganhar tráfego aberto, é o primeiro item a acrescentar; a contenção natural seria limitar por IP na camada do Cloudflare, que já está no caminho.

PARTE VI

Interface e sistema de design

25 Paleta e tipografia

Os dois tons âncora foram **amostrados da logo oficial**, não estimados: o petróleo do lettering (#0a292b) e o laranja do coração (#ed9216). O restante da escala foi derivado deles.

TAILWIND.CONFIG.TS — AS DUAS ESCALAS DA MARCA

```
1      petroleo: {
2        50: '#f0f7f6',
3        100: '#d9ecea',
4        200: '#b4d8d5',
5        300: '#83bbb7',
6        400: '#519a96',
7        500: '#2e7d79',
8        600: '#1f6362',
9        700: '#1a4f50',
10       800: '#143d3f',
11       900: '#0a292b',
12     },
13     laranja: {
14       50: '#fff8ec',
15       100: '#ffedcf',
16       200: '#ffd893',
17       300: '#ffbe57',
18       400: '#f9a62c',
19       500: '#ed9216',
20       600: '#d1780c',
21       700: '#a85a0d',
22       800: '#874712',
23       900: '#6f3b12',
24     },
```

REGRA DA MARCA

O **petróleo carrega estrutura e texto**; o **laranja carrega afeto e ação**. Laranja aparece em botão de doar, em barra de meta e em número de arrecadação — e em mais nada. Quando tudo é laranja, nada é, e o botão que precisa converter perde a vez.

Complementam a paleta uma escala neutra (tinta , tinta-suave , tinta-clara , tinta-borda , tinta-fundo) e três cores de estado: alerta , risco e seguro .

A tipografia é a pilha do sistema operacional, sem fonte externa. O motivo é concreto: a página pública do animal é aberta a partir de um link do WhatsApp, quase sempre no celular e muitas vezes em 4G ruim. Uma fonte remota adiciona uma rodada de rede antes do primeiro texto legível.

26 Componentes

26.1 Classes de componente

Oito classes em `globals.css` concentram o vocabulário visual: `.cartao`, `.campo`, `.rotulo`, `.botao` com quatro variantes (`-doar`, `-primario`, `-secundario`, `-fantasma`), `.selo`, `.titulo-seccao`, `.esqueleto` e `.patinhas`.

Dois detalhes de toque valem registro. Os botões têm altura mínima de 44 px — o alvo abaixo disso erra no celular, e o app é usado em pé, com uma mão, às vezes com o animal no colo. E todos trazem `touch-action: manipulation`: sem isso o navegador espera 300 ms para saber se foi duplo-toque, e o aplicativo parece lento mesmo respondendo na hora.

26.2 Peças compartilhadas

COMPONENTE	PAPEL
BarraMeta	A peça mais repetida do produto: vitrine, perfil, campanha, painel e área do padrinho. Tem <code>role="progressbar"</code> com valores ARIA, e o texto acompanha sempre a barra — quem não distingue a cor, quem imprime em preto e branco e quem usa leitor de tela leem o mesmo número. Barra sem número é decoração.
Indicador	Número grande com rótulo. Usa <code>tabular-nums</code> para que uma fila de cartões não dance quando os números mudarem de largura.
FotoAnimal / RetratoGerado	Foto real ou, enquanto ela não existe, um retrato SVG gerado a partir do nome — gradiente determinístico, patinha deslocada e inicial. A vitrine é a captação da ONG, e um cartão cinza no meio da grade mata a fileira inteira.
Navegacao	Coluna no desktop; no celular, uma tira rolável de pílulas. Não é barra inferior de cinco abas porque a administração tem doze destinos — cortar para cinco e esconder sete num “mais” seria pior do que deixar rolar.
Link	Envolve o <code>next/link</code> e acrescenta um fio de progresso via <code>useLinkStatus</code> . Numa lista de noventa animais cada navegação é uma ida ao servidor; sem sinal nenhum, o toque parece não ter funcionado e a pessoa toca de novo.
Compartilhar	Usa <code>navigator.share</code> no celular — que é o caminho real: a pessoa abre o link, gosta, manda no grupo da família. No desktop copia a mensagem pronta, com texto e não só a URL, porque link solto no WhatsApp não convence ninguém. Se a área de transferência falhar, abre o WhatsApp Web com a mensagem montada.

27 Movimento e acessibilidade

Duas curvas de animação no sistema inteiro, e só duas: **entrada desacelera** (o elemento chega e para) e **saída acelera** (some sem pedir atenção). Mais do que isso vira sotaque inconsistente.

Seis animações nomeadas cobrem todo o movimento: `sobe`, `entra-tela`, `brilho` (esqueleto de carregamento), `barra-rota`, `enche` (a barra de meta crescendo da esquerda) e `batida`.

A barra de progresso da rota nunca chega a 100% sozinha — vai até 94% e para. Quem a fecha é a tela nova entrando. Uma barra que completa e continua esperando mente para o usuário.

PREFERS-REDUCED-MOTION

A regra que desliga o movimento é a **última** do `globals.css`, de propósito, para vencer tudo que está acima — inclusive a barra de meta, que é a animação mais insistente do sistema. Quem pediu menos movimento recebe menos movimento: trocar acessibilidade por acabamento não se paga.

Outras garantias de acessibilidade: `aria-current` nos itens de navegação ativos, `role="status"` nos avisos e contagens de resultado, rótulo associado a todo campo de formulário, `viewport` sem `maximumScale` (quem quiser ampliar a foto do animal precisa poder) e área segura do iPhone respeitada no rodapé.

28 Gráficos

Quatro formas, escritas à mão em SVG: barras agrupadas, linha com área, rosca e medidor. Cerca de 500 linhas em `src/components/Graficos.tsx`, contra as dezenas de kB de JavaScript que qualquer biblioteca traria para o celular de quem só quer ver quanto entrou no mês.

28.1 A paleta categórica, validada

SRC/LIB/PALETA.TS

```
1 export const CORES = {
2   entrada: '#e8860a',
3   saida: '#008a7c',
4   serie: ['#008a7c', '#e8860a', '#c33a6b', '#79a017'] as const,
5   neutro: '#8c9895',
6   grade: '#e2eae8',
7 };
8
9 const CORES_CATEGORIA: Record<string, string> = {
10  VETERINARIO: '#008a7c',
11  ALIMENTACAO: '#e8860a',
12  MEDICAMENTO: '#c33a6b',
13  HIGIENE: '#79a017',
14  TRANSPORTE: '#1a4f50',
15  ESTRUTURA: '#a85a0d',
16  OUTRO: '#8c9895',
17 };
18
19 export function corDaCategoria(categoria: string): string {
20   return CORES_CATEGORIA[categoria] ?? CORES.neutro;
21 }
```

Os quatro tons não foram escolhidos por gosto. Passaram por um validador de paleta que confere faixa de luminosidade, piso de croma, separação em três tipos de daltonismo e contraste contra a superfície. O resultado:

VERIFICAÇÃO	RESULTADO	MEDIDA
Faixa de luminosidade	passa	os 4 tons dentro de L 0,43-0,77
Piso de croma	passa	todos $\geq 0,1$ — nenhum lê como cinza
Separação em daltonismo	passa	pior par adjacente ΔE 13,1 em deuteranopia
Piso de visão normal	passa	pior par adjacente ΔE 22,3
Contraste com o fundo	<i>aviso</i>	laranja 2,61:1 e verde 2,98:1 — abaixo de 3:1

COMO O AVISO DE CONTRASTE É COMPENSADO

O aviso não é dispensável: ele obriga um alívio. Por isso **todo gráfico do sistema traz legenda com rótulo em texto e uma tabela equivalente logo abaixo**, recolhida num `<details>` chamado “ver os números”. Cor nunca é a única forma de ler um número aqui — o que atende leitor de tela, impressão em preto e branco e quem só quer o valor exato.

28.2 Anatomia das marcas

- **Grade recessiva:** três ou quatro linhas, 1 px, cor de borda. Ela orienta, não decora.
- **Folga entre marcas:** 2 unidades entre barras vizinhas e entre fatias da rosca. Sem isso, duas cores encostadas viram uma só à distância.
- **Anel branco** no ponto ativo da linha, para que ele sobreviva por cima da área preenchida.
- **Alvo de mouse maior que a marca:** o retângulo transparente cobre a coluna inteira, para que o balão não pisque entre uma barra e outra.
- **Regra do eixo único:** nunca dois eixos y no mesmo gráfico. Quando as grandezas têm escalas diferentes, viram dois gráficos.
- **Cor segue a entidade, não a posição:** “arrecadado” é sempre laranja e “despesas” é sempre petróleo, em toda tela onde aparecem.

28.3 Onde cada forma aparece

FORMA	USO
GraficoBarras	Arrecadação × despesas (painel, financeiro, impacto); custeio por categoria; novos × cancelados; cobrado × recebido.
GraficoLinha	Evolução de uma grandeza só, com área e cursor.
GraficoDonut	Composição das despesas do mês e receita por modalidade. A legenda traz valor e percentual, porque rótulo dentro da fatia, a 40 px de raio, viraria sopa de letras.
Medidor	Estoque de cada item. O teto é o dobro do mínimo — com o teto no próprio mínimo, qualquer estoque saudável encostaria em 100% e a barra pararia de informar.

PARTE VII

Meu AUmigo AI

29 O assistente interno

Uma tela de conversa em `/ia`, exclusiva da equipe, que usa a API da Claude para quatro tipos de tarefa: **resumir** (histórico médico, situação do mês), **analisar** (onde a despesa cresceu, quais animais concentram custo), **escrever** (atualização para padrinhos, texto de campanha, post de Instagram, prestação de contas) e **responder** perguntas sobre os dados já cadastrados.

29.1 O que ela não faz

LIMITE	COMO É IMPOSTO
Não publica nada	A tela devolve texto para copiar; não há botão de enviar ao padrinho. Todo texto passa pela equipe antes de virar comunicação.
Não consulta o banco sozinha	Não recebe ferramenta de leitura. Vê apenas o resumo montado no servidor, de tamanho conhecido — o que dá custo previsível e evita que uma pergunta malformada varra a base.
Não dá orientação clínica	Proibição explícita no prompt de sistema. Dose, diagnóstico e tratamento são do veterinário responsável.
Não compromete a ONG	Proibida de escrever garantias, promessas de devolução ou compromisso de data em nome da organização.
Não é caminho crítico	Sem <code>ANTHROPIC_API_KEY</code> , a tela mostra um aviso de “não configurado” e todo o resto do sistema funciona igual . A flag <code>iaDisponivel</code> é derivada no boot.

29.2 O contexto enviado

SRC/SERVER/IA/CONTEXTO.TS — MONTAGEM DO RETRATO

```
1 export async function montarContexto(): Promise<string> {
2   const mes = inicioDoMes();
3   const proximo = fimDoMes();
4
5   const [animais, entradas, despesasCat, porAnimal, campanhas, estoque, semPadrinho, atrasadas] =
6     await Promise.all([
7       db.animal.findMany({
8         where: { ativo: true, deletadoEm: null },
9         orderBy: { dataResgate: 'desc' },
10        select: {
11          nome: true,
12          slug: true,
13          status: true,
14          especie: true,
15          porte: true,
16          dataResgate: true,
17          necessidadeEspecial: true,
18          ...selecaoCusto,
19        },
20      }),
21      db.contribuicao.aggregate({
22        where: { status: 'PAGA', competencia: { gte: mes, lt: proximo } },
23        _sum: { valor: true },
24        _count: true,
25      }),
26      db.despesa.groupBy({
27        by: ['categoria'],
28        where: { data: { gte: mes, lt: proximo } },
29        _sum: { valor: true },
30      }),
31      db.despesa.groupBy({
32        by: ['animalId'],
33        where: { data: { gte: mes, lt: proximo }, animalId: { not: null } },
34        _sum: { valor: true },
```

O retrato reúne, em texto: os animais ativos com status, custo, cobertura e número de padrinhos; o financeiro do mês (arrecadado, despesas por categoria, valores em aberto); as maiores despesas por animal; as campanhas ativas com meta e arrecadado; e o estoque com marcação de item abaixo do mínimo.

PRIVACIDADE

Nome, e-mail, telefone e documento de padrinho **não entram no contexto**. Nomes de animais e agregados financeiros, sim. Quem doa não é assunto de modelo de terceiro; quanto entrou, é.

29.3 O prompt de sistema

SRC/SERVER/IA/ASSISTENTE.TS — INSTRUÇÃO (INÍCIO)

```
1 const INSTRUCAO = `Você é o assistente interno do Meu AUmigo, um sistema de apadrinhamento de animais
2 resgatados usado pela equipe de uma ONG brasileira.
3 Você recebe, a cada pergunta, um retrato atual dos dados da ONG. Responda SEMPRE com base nesse
4 retrato.
5 Regras:
6 - Responda em português do Brasil, direto, sem rodeio e sem elogiar a pergunta.
7 - Use os números do contexto. Se o dado não estiver lá, diga que não está – nunca estime nem invente
8 valor, nome de animal ou data.
9 - Valores em reais no formato brasileiro (R$ 1.234,56).
10 - Quando listar animais, use o nome deles.
11 - Texto para padrinho ou para rede social: escreva pronto para copiar, caloroso mas sem pieguice, sem
12 prometer cura nem resultado.
13 - Nunca dê orientação clínica, dose de medicamento ou diagnóstico: isso é do veterinário responsável.
14 - Nunca escreva em nome da ONG algo que comprometa dinheiro ou prazo (garantias, promessas de
15 devolução, compromisso de data).
16 - Seja curto. Se a resposta cabe em três linhas, use três linhas.`;
```

Duas escolhas de implementação sustentam custo e qualidade:

- **Dois blocos de sistema separados.** A instrução é estável; o retrato dos dados muda a cada pergunta. Mantê-los em blocos distintos é o que permite ao cache de prompt fazer efeito sobre a parte fixa.
- **Janela de dez mensagens.** O histórico enviado é truncado em `slice(-10)`: uma conversa longa não cresce indefinidamente em custo, e o contexto relevante é o retrato atual, não o que se falou há vinte turnos.

Falha de rede ou de API é capturada, registrada no log do container e devolve uma mensagem que diz exatamente o que continua funcionando — “o resto do sistema segue funcionando normalmente”. A rota responde 503 nesse caso, não 500: é indisponibilidade de dependência externa, não erro da aplicação.

PARTE VIII

Infraestrutura

30 A imagem

Build em três estágios. O `deps` instala dependências, o `builder` gera o cliente Prisma e compila o Next, e o `runner` sai enxuto:

DOCKERFILE — ESTÁGIO RUNNER

```
1 FROM node:20-slim AS runner
2 WORKDIR /app
3
4 # openssl: exigido pelo engine do Prisma. wget: healthcheck do compose.
5 RUN apt-get update && apt-get install -y --no-install-recommends openssl wget \
6     && rm -rf /var/lib/apt/lists/*
7
8 ENV NODE_ENV=production
9 ENV NEXT_TELEMETRY_DISABLED=1
10
11 RUN groupadd -g 1001 nodejs && useradd -u 1001 -g nodejs -m aumigo
12
13 COPY --from=builder /app/public ./public
14 COPY --from=builder --chown=aumigo:nodejs /app/.next/standalone ./
15 COPY --from=builder --chown=aumigo:nodejs /app/.next/static ./next/static
16
17 # node_modules completo, prisma/ e scripts/: o entrypoint roda
18 # `migrate deploy`, os GRANTS e a semente de demonstração.
19 COPY --from=builder /app/node_modules ./node_modules
20 COPY --from=builder /app/prisma ./prisma
21 COPY --from=builder /app/scripts ./scripts
22 COPY --from=builder /app/src ./src
23 COPY --from=builder /app/tsconfig.json ./tsconfig.json
24 COPY docker-entrypoint.sh /usr/local/bin/docker-entrypoint.sh
25 RUN chmod +x /usr/local/bin/docker-entrypoint.sh
26
27 # Fotos e vídeos dos animais, comprovantes de despesa.
28 RUN mkdir -p /app/storage/animais /app/storage/comprovantes \
29     && chown -R aumigo:nodejs /app/storage
30
31 USER aumigo
32 EXPOSE 3000
33
34 ENTRYPOINT ["docker-entrypoint.sh"]
35 CMD ["node", "server.js"]
```

Pontos de atenção:

- **Usuário não-root.** O processo roda como `aumigo` (uid 1001), e o volume de mídia é criado já com o dono correto.
- **openssl é obrigatório:** o engine do Prisma não sobe sem ele. `wget` existe só para o healthcheck do compose.
- **O `node_modules` completo é copiado,** apesar da saída `standalone` do Next. É proposital: o entrypoint roda `prisma migrate deploy`, executa o `grants.sql` e chama a semente com `tsx` — nada disso está no bundle da aplicação.

- **Valor fictício de `DATABASE_URL` no build.** O estágio de compilação só precisa que o Prisma gere o cliente; nada ali alcança banco de verdade.

31 Compose e a rede

Um serviço, um volume, uma rede. A parte que mais causa incidente nesta VPS é a rede:

DOCKER-COMPOSE.YML — A REDE

```
1 networks:
2   meuaumigo-net:
3     driver: bridge
4     ipam:
5       config:
6         # Subnet FIXA para casar com a regra de ufw que libera a 5432 do
7         # Postgres do host. Sem fixar, o Docker sorteia outra faixa ao
8         # recriar e a regra para de valer. Ver: `ufw status | grep 5432`.
9         - subnet: 172.16.61.0/24
10        gateway: 172.16.61.1
```

POR QUE A SUBNET É FIXA

O PostgreSQL roda no host, atrás do `ufw`. Para o container alcançá-lo, existe uma regra que libera a porta 5432 **para aquela faixa de IP**. Se a subnet não for fixada no compose, o Docker sorteia outra faixa ao recriar a rede e a regra para de valer — silenciosamente, e só na próxima recriação. O sintoma é `P1001: Can't reach database server` num container que subiu sem erro aparente.

A regra correspondente, no host:

```
ufw allow from 172.16.61.0/24 to any port 5432 proto tcp comment 'meuaumigo docker network'
```

Outros pontos do compose:

CONFIGURAÇÃO	RAZÃO
<code>extra_hosts:</code> <code>host.docker.internal:host-gateway</code>	É assim que o container resolve o endereço do host. De dentro do container, o banco nunca é <code>localhost</code> .
<code>TZ: America/Sao_Paulo</code>	A imagem base roda em UTC. Sem isso, competência, data de resgate e “arrecadado este mês” saem três horas adiantados — e no dia 1º isso muda o mês inteiro de lugar.
<code>healthcheck</code> em <code>/api/health</code>	Bate no banco de propósito: um container que responde 200 sem conseguir ler o Postgres está “no ar” e inútil — e é justamente esse o modo de falha desta máquina.
<code>volumes: midia:/app/storage</code>	Fotos, vídeos e comprovantes. Entra na rotina de backup com a mesma prioridade do banco: a foto do animal é metade da captação.
<code>logging</code> com rotação	10 MB por arquivo, 3 arquivos. Container sem limite de log enche disco compartilhado.

32 Entrypoint

DOCKER-ENTRYPOINT.SH

```
1 #!/bin/sh
2 set -e
3
4 # =====
5 # Boot da aplicação.
6 #
7 # Migrations e GRANTS rodam com o papel DONO, nunca com o da aplicação:
8 # a app não pode ter permissão de alterar o próprio schema.
9 # =====
10
11 echo "→ Aplicando migrations..."
12 DATABASE_URL="${DATABASE_URL_OWNER:-$DATABASE_URL}" npx prisma migrate deploy
13
14 echo "→ Aplicando GRANTS..."
15 DATABASE_URL="${DATABASE_URL_OWNER:-$DATABASE_URL}" \
16   npx prisma db execute --file prisma/sql/grants.sql --schema prisma/schema.prisma
17
18 echo "→ Semeando (idempotente: não toca em banco que já tem animal)..."
19 node_modules/.bin/tsx prisma/seed.ts || echo "  aviso: semente falhou, seguindo"
20
21 exec "$@"
```

Três passos, nesta ordem, e todos com o papel **dono**: migrações, privilégios e semente. A semente falha sem derrubar o boot — ela é conveniência, não requisito.

Reaplicar os GRANTS a cada subida é intencional: garante que tabela criada por migração futura já nasce acessível à aplicação, e desfaz qualquer afrouxamento feito à mão no banco.

33 Variáveis de ambiente

Todas passam por um schema Zod no boot. Segredo ausente derruba o processo na partida, em vez de falhar no meio de um apadrinhamento — que é o momento exato em que ninguém tenta de novo.

SRC/LIB/ENV.TS — O SCHEMA DO AMBIENTE

```
1 const schema = z.object({
2   NODE_ENV: z.enum(['development', 'test', 'production']).default('development'),
3
4   DATABASE_URL: z.string().min(1),
5
6   STORAGE_DIR: z.string().default('/app/storage'),
7
8   APP_URL: z.string().url().default('http://localhost:3000'),
9   SESSAO_HORAS: z.coerce.number().int().min(1).max(72).default(12),
10
11   // Sem chave, a tela da IA mostra "não configurado" e o resto do sistema
12   // funciona igual. A IA nunca é caminho crítico.
13   ANTHROPIC_API_KEY: z.string().optional(),
14   MODELO_IA: z.string().default('claude-sonnet-5'),
15
16   SEMEAR_DEMONSTRACAO: z
17     .enum(['true', 'false'])
18     .default('false')
19     .transform((v) => v === 'true'),
20 });
```

VARIÁVEL	OBRIGATÓRIA	DESCRIÇÃO
NODE_ENV	não	<code>production</code> no container. Afeta log do Prisma e o <code>Secure</code> do cookie.
APP_PORT	não	Porta publicada no host. Padrão 1046. Lida pelo compose, não pela aplicação.
APP_URL	não	URL pública. Usada nas tags Open Graph, nos textos de compartilhamento e na checagem de origem. Precisa ser ajustada quando o domínio for definido.
DATABASE_URL	sim	Conexão da aplicação, com o papel <code>meuaumigo_app</code> . Host sempre <code>host.docker.internal</code> .
DATABASE_URL_OWNER	sim*	Conexão do dono do schema. Usada apenas pelo entrypoint. Se ausente, o entrypoint cai para a anterior — e as migrações falham, como devem.
STORAGE_DIR	não	Raiz da mídia. Padrão <code>/app/storage</code> , montado como volume.
SESSAO_HORAS	não	Validade da sessão, entre 1 e 72. Padrão 12.
ANTHROPIC_API_KEY	não	Habilita a tela de IA. Ausente, a tela informa e o resto segue.
MODELO_IA	não	Padrão <code>claude-sonnet-5</code> .
SEMEAR_DEMONSTRACAO	não	<code>true</code> popula o banco vazio com dados fictícios. Virar para <code>false</code> quando o primeiro dado real entrar.

34 Cabeçalhos e indexação

Em `next.config.mjs`, dois conjuntos de cabeçalhos com propósitos opostos:

- **Para todas as rotas:** `X-Content-Type-Options: nosniff`, `Referrer-Policy: strict-origin-when-cross-origin` e `Permissions-Policy` negando câmera, geolocalização e microfone — o sistema não usa nenhum dos três.
- **Só para a área logada:** `X-Robots-Tag: noindex, nofollow`. Note que as páginas públicas dos animais *não* recebem essa marcação, e isso é deliberado — é de busca orgânica que vem padrinho novo.

Não há `X-Frame-Options: DENY` global, ao contrário de outros sistemas desta VPS: a página do animal precisa poder ser embutida por agregadores e parceiros. A área logada está protegida pelo `SameSite` do cookie de sessão.

O otimizador de imagem do Next está desligado (`images.unoptimized`): as fotos vêm do próprio endpoint da aplicação, já com cache imutável, e passar por um otimizador acrescentaria processamento sem ganho.

PARTE IX

Dados de demonstração

35 A semente

O sistema chega à mão da equipe antes de existir dado verdadeiro, e painel vazio não ensina nada: ninguém entende um gráfico de arrecadação olhando para zero, nem avalia se a tela de financeiro serve quando ela não tem uma linha. A semente cria **doze meses de história fictícia**.



Mais 3 campanhas, 12 processos de adoção, 138 notificações, 6 itens de estoque e 3 veterinários.

35.1 As duas salvaguardas

PRISMA/SEED.TS — A TRAVA

```
1  const jaTemAnimal = await db.animal.count();
2  if (jaTemAnimal > 0) {
3    console.log(` já existem ${jaTemAnimal} animais cadastrados – a demonstração não roda.`);
4    return;
5  }
```

1. **Se já houver qualquer animal cadastrado, a demonstração não roda.** Dado de verdade nunca é sobrescrito. Essa checagem acontece a cada boot do container, porque o entrypoint chama a semente sempre.
2. **SEMEAR_DEMONSTRACAO=false desliga de vez.** É o interruptor a virar no dia em que o primeiro animal real for cadastrado.

Os **catálogos base** — configuração da ONG e os três acessos de equipe — são semeados sempre, de forma idempotente por `upsert`. Eles não são demonstração: são o mínimo para alguém conseguir entrar no sistema.

35.2 Por que os dados são realistas

Dados fictícios servem para testar a interface, e interface só é testada por dados que se parecem com os reais. A semente foi escrita com isso em mente:

CARACTERÍSTICA	POR QUÊ
Gerador determinístico	Um PRNG linear com semente fixa (20260922). O mesmo abrigo é gerado toda vez; sem isso, cada deploy mostraria números diferentes e ninguém conseguiria conferir se algo quebrou.
~9% de inadimplência	Um sistema com 100% de pagamento não exercita a tela de cobrança nem o alerta do painel.
~12% de cancelamento	Dá ao gráfico de retenção o que mostrar. Um gráfico de novos x cancelados sem cancelamento não ensina a ler o gráfico.
Mês corrente parcialmente pago	62% pago, o resto pendente ou atrasado conforme o vencimento — que é exatamente o estado real de um dia 22.
Poucas cirurgias caras	7% dos animais têm um procedimento acima de R\$ 1.200. É o que justifica a existência de campanha e o que faz o relatório de concentração de custo ter sentido.
Histórias de resgate escritas	Dez textos reais de tom, não <i>lorem ipsum</i> . É esse texto que faz alguém parar o scroll, e testar a vitrine com texto falso não testa nada.
Coerência temporal	Nenhum apadrinhamento começa antes do resgate do animal ou do cadastro do padrinho; nenhum cancelamento tem data futura; nenhuma saída de estoque deixa saldo negativo.

SEM FOTOS, E DE PROPÓSITO

A semente não cria arquivos de imagem. Os animais aparecem com o **retrato gerado** — gradiente determinístico com a inicial do nome —, o que também serve para validar esse caminho da interface. Os comprovantes fictícios existem como metadado, mas o arquivo não: abrir um deles devolve 404, e é melhor assim — comprovante de mentira não deve abrir como se fosse documento de verdade.

35.3 Acessos de demonstração

E-MAIL	PAPEL
admin@meuaumigo.com.br	Direção — vê financeiro e configurações
rita@meuaumigo.com.br	Equipe
padrinho@meuaumigo.com.br	Área do padrinho

A SENHA NÃO ESTÁ NO CÓDIGO

A senha dos três vem de `SENHA_DEMO`, no `.env`. Deixando a variável em branco, a semente **sorteia** uma e a imprime uma única vez no log do boot — quem subiu o container a lê com `docker compose logs app`, e mais ninguém.

O motivo é direto: o repositório é público, e senha de administrador versionada é senha de administrador publicada. Vale a mesma regra para o `.env.example`, que vai para o Git.

ANTES DE EXPOR O SISTEMA

Troque as senhas dos três acessos, pela tela de configurações ou por `scripts/criar-usuario.ts`. Uma senha sorteada no boot é melhor do que uma senha publicada, mas continua sendo uma senha de demonstração.

PARTE X

Operação

36 Deploy

SUBIR OU ATUALIZAR

```
cd /srv/meuaumigo # onde o projeto foi clonado
docker compose build
docker compose up -d
docker compose logs -f app
```

O entrypoint aplica as migrações pendentes, reaplica os privilégios e chama a semente (que não faz nada se já houver animal cadastrado). O healthcheck leva até 40 segundos para o primeiro sinal verde.

36.1 Instalação em máquina nova

BANCO — UMA VEZ, NO HOST

```
CREATE ROLE meuaumigo_owner LOGIN PASSWORD '...';
CREATE ROLE meuaumigo_app LOGIN PASSWORD '...';
CREATE DATABASE meuaumigo OWNER meuaumigo_owner;
REVOKE ALL ON DATABASE meuaumigo FROM PUBLIC;
GRANT CONNECT ON DATABASE meuaumigo TO meuaumigo_app;
-- conectado em meuaumigo:
ALTER SCHEMA public OWNER TO meuaumigo_owner;
REVOKE ALL ON SCHEMA public FROM PUBLIC;
```

FIREWALL E APLICAÇÃO

```
ufw allow from 172.16.61.0/24 to any port 5432 proto tcp comment 'meuaumigo docker network'
cp .env.example .env # preencher DATABASE_URL e DATABASE_URL_OWNER
docker compose up -d --build
```

37 Rotina mensal da equipe

Não há agendador. Todo mês, em `/financeiro`:

1. **Gerar mensalidades do mês** — cria um lançamento pendente por apadrinhamento ativo. Idempotente: pode ser clicado duas vezes sem duplicar.
2. **Marcar vencidas como atrasadas** — move para `ATRASADA` o que passou do vencimento e continua pendente.
3. **Conferir o extrato** e confirmar cada pagamento, escolhendo a forma. A confirmação grava `pagoEm` e alimenta o painel, os relatórios e o rateio que o padrinho vê.

Ao longo do mês, o trabalho do dia a dia acontece na ficha de cada animal: a novidade para os padrinhos, o registro de saúde, o marco do diário e a foto. Esses quatro estão juntos num painel de abas justamente porque são o que a equipe faz todo dia, e quase sempre pelo celular.

38 Backup

DUAS COISAS, MESMA PRIORIDADE

```
# banco
pg_dump -h localhost -U meuaumigo_owner meuaumigo | gzip > meuaumigo-$(date +%F).sql.gz

# mídia – fotos dos animais e comprovantes de despesa
docker run --rm -v meuaumigo_midia:/dados -v "$PWD":/saida alpine \
  tar czf /saida/meuaumigo-midia-$(date +%F).tar.gz -C /dados .
```

NÃO PULE O SEGUNDO

A foto do animal é metade da captação da ONG. Um banco restaurado sem as fotos devolve um sistema que ninguém quer usar — noventa cartões com retrato gerado e nenhuma história visual.

39 Diagnóstico

39.1 Os três erros recorrentes desta VPS

SINTOMA	CAUSA E CORREÇÃO
P1001: Can't reach database server no boot	1. Host errado no .env . De dentro do container o Postgres é <code>host.docker.internal</code> , nunca <code>localhost</code> . Confira com <code>docker compose exec app printenv DATABASE_URL</code> .
Mesmo sintoma, .env correto	2. Firewall não liberado. <code>ufw status grep 172.16.61</code> — se não aparecer, aplique a regra da seção 36.1.
Funcionava e parou depois de recriar	3. Subnet mudou. Alguém removeu a fixação em <code>ipam.config.subnet</code> e o Docker sorteou outra faixa. A regra do ufw aponta para a faixa antiga. Restaure a fixação e recrie a rede.

39.2 Outros diagnósticos

SINTOMA	ONDE OLHAR
Container <code>unhealthy</code>	<code>curl localhost:1046/api/health</code> . A rota executa <code>SELECT 1</code> ; se devolver 503, o problema é banco, não aplicação.
Datas e valores com três horas de diferença	<code>TZ</code> ausente no compose. Confira com <code>docker compose exec app date</code> .
Tela de IA diz “não configurado”	<code>ANTHROPIC_API_KEY</code> vazia. É comportamento esperado, não erro.
Foto some depois de recriar o container	O volume <code>meuaumigo_midia</code> foi removido. <code>docker volume ls grep meuaumigo</code> .
Ninguém consegue entrar	Use o script de emergência (abaixo). Ele reativa a conta junto, porque quem o executa quase sempre está destrancando a porta.

SINTOMA	ONDE OLHAR
Failed to find Server Action	Aba antiga aberta durante um deploy. O identificador da ação muda a cada build; basta recarregar.

DESTRANCAR O ACESSO

```
docker compose exec app node_modules/.bin/tsx scripts/criar-usuario.ts \  
  "Nome da Pessoa" email@ong.com.br senhaSegura ADMIN
```

39.3 Onde as coisas estão

ITEM	VALOR
Diretório do projeto	onde o repositório foi clonado no servidor
Container / imagem	meuaumigo-app / meuaumigo:latest
Porta publicada	1046 (Cloudflare Tunnel aponta para ela)
Rede / subnet	meuaumigo-net / 172.16.61.0/24
Volume de mídia	meuaumigo_midia → /app/storage
Banco	meuaumigo , no PostgreSQL do host
Papéis	meuaumigo_app · meuaumigo_owner

APÊNDICES

Referência

A Referência de arquivos

Todos os arquivos de código do projeto, com a contagem de linhas e o papel de cada um. Somam 15.454 linhas em 116 arquivos; `node_modules` , artefatos de build e mídia não entram na conta.

Banco de dados

ARQUIVO	LINHAS	PAPEL
<code>prisma/migrations/0001_init/migration.sql</code>	608	Migração inicial gerada pelo Prisma, versionada
<code>prisma/schema.prisma</code>	728	20 modelos, 20 enums — a fonte da verdade do modelo de dados
<code>prisma/seed.ts</code>	984	Semente de demonstração: 12 meses de história fictícia, com duas salvaguardas
<code>prisma/sql/grants.sql</code>	31	Privilégios do papel da aplicação; idempotente, roda a cada deploy

Infraestrutura da aplicação (src/lib)

ARQUIVO	LINHAS	PAPEL
<code>src/lib/auth.ts</code>	187	Sessão, os quatro portões de papel, CSRF e auditoria
<code>src/lib/crypto.ts</code>	28	Argon2id para senha; token de sessão e seu hash SHA-256
<code>src/lib/db.ts</code>	18	Prisma Client único, com o papel de aplicação
<code>src/lib/env.ts</code>	38	Ambiente tipado com Zod; segredo ausente derruba o boot
<code>src/lib/format.ts</code>	361	Dinheiro, datas, idade, faixa etária e os rótulos de todos os enums
<code>src/lib/paleta.ts</code>	38	Cores dos gráficos — fora do módulo de cliente, de propósito
<code>src/lib/slug.ts</code>	28	Endereço público do animal; sufixo numérico para nomes repetidos
<code>src/lib/storage.ts</code>	77	Mídia em disco; nome de arquivo sempre gerado pelo sistema

Regras de negócio (src/server)

ARQUIVO	LINHAS	PAPEL
src/server/config.ts	28	Configuração da ONG, memoizada por requisição
src/server/custos.ts	137	Cobertura da meta e o rateio da contribuição por categoria
src/server/gamificacao.ts	75	Conquistas derivadas dos apadrinhamentos, nunca armazenadas
src/server/ia/assistente.ts	79	Prompt de sistema, limites e a chamada à API da Claude
src/server/ia/contexto.ts	134	Monta o retrato agregado que vai ao modelo; sem dado pessoal
src/server/padrinho.ts	159	A área do padrinho: animais, rateio, conquistas e notificações
src/server/painel.ts	266	Agregados do painel (um Promise.all) e os números de impacto
src/server/periodo.ts	33	Competência mensal — o único lugar que decide onde um mês começa
src/server/publico.ts	197	Vitrine com filtros e o perfil público do animal
src/server/relatorios.ts	138	Retenção, cobrado × recebido, concentração de custo, descobertos
src/server/transparencia.ts	86	Prestação de contas pública; só o que foi marcado como público

Componentes compartilhados

ARQUIVO	LINHAS	PAPEL
src/components/BarraUsuario.tsx	29	Identidade do usuário e a ação de sair
src/components/CartaoAnimal.tsx	79	O cartão da vitrine pública
src/components/Compartilhar.tsx	58	navigator.share, com queda para área de transferência e WhatsApp
src/components/FotoAnimal.tsx	91	Foto real ou retrato SVG gerado a partir do nome
src/components/Graficos.tsx	478	Barras, linha, rosca e medidor em SVG, com hover e tabela equivalente
src/components/Icones.tsx	237	23 ícones desenhados à mão, traço 1,8
src/components/Link.tsx	33	next/link com fio de progresso via useLinkStatus
src/components/Marca.tsx	42	Logo, em versão completa e compacta
src/components/Navegacao.tsx	146	Lateral no desktop; tira rolável de pílulas no celular

ARQUIVO	LINHAS	PAPEL
<code>src/components/ui.tsx</code>	217	Cartao, TituloPagina, Indicador, BarraMeta, Tabela, Aviso, Linha, Vazio

Site público

ARQUIVO	LINHAS	PAPEL
<code>src/app/(auth)/entrar/Formulario.tsx</code>	50	Formulário com useActionState
<code>src/app/(auth)/entrar/actions.ts</code>	66	entrar() e sair(); mensagem única e tempo constante
<code>src/app/(auth)/entrar/page.tsx</code>	30	Tela de login; redireciona quem já tem sessão
<code>src/app/(auth)/layout.tsx</code>	13	Moldura do login, com fundo de patinhas
<code>src/app/(publico)/adotar/[slug]/Formulario.tsx</code>	125	Moradia, outros animais e rotina
<code>src/app/(publico)/adotar/[slug]/actions.ts</code>	90	manifestarInteresse(): abre processo em INTERESSE
<code>src/app/(publico)/adotar/[slug]/page.tsx</code>	100	Formulário de interesse; recusa se indisponível
<code>src/app/(publico)/animais/[slug]/apadrinhar/Formulario.tsx</code>	219	Modalidades em cartões e os dados do padrinho
<code>src/app/(publico)/animais/[slug]/apadrinhar/actions.ts</code>	171	apadrinhar(): upsert do padrinho e a transação
<code>src/app/(publico)/animais/[slug]/apadrinhar/page.tsx</code>	99	Página de apadrinhamento com o resumo do custo
<code>src/app/(publico)/animais/[slug]/page.tsx</code>	391	Perfil público: história, galeria, saúde, diário, meta
<code>src/app/(publico)/animais/page.tsx</code>	212	Vitrine com busca e seis grupos de filtro em links
<code>src/app/(publico)/campanhas/[slug]/Formulario.tsx</code>	116	Doação com valores sugeridos
<code>src/app/(publico)/campanhas/[slug]/actions.ts</code>	74	doar(): registra como PENDENTE
<code>src/app/(publico)/campanhas/[slug]/page.tsx</code>	151	Página da campanha com meta e compartilhamento
<code>src/app/(publico)/campanhas/page.tsx</code>	119	Campanhas ativas e as que deram certo
<code>src/app/(publico)/impacto/page.tsx</code>	158	Transparência: impacto, fluxo, composição e comprovantes
<code>src/app/(publico)/layout.tsx</code>	116	Cabeçalho, navegação e rodapé do site público

ARQUIVO	LINHAS	PAPEL
<code>src/app/(publico)/page.tsx</code>	227	Home: conceito, campanha urgente, quem espera, impacto

Área do padrinho

ARQUIVO	LINHAS	PAPEL
<code>src/app/(padrinho)/layout.tsx</code>	35	Moldura da área do padrinho
<code>src/app/(padrinho)/meus-aumigos/[slug]/page.tsx</code>	308	A tela central: como o animal está e onde vai o apoio
<code>src/app/(padrinho)/meus-aumigos/conquistas/page.tsx</code>	92	Tempo de amizade e conquistas
<code>src/app/(padrinho)/meus-aumigos/contribuicoes/page.tsx</code>	129	Histórico de pagamentos e a chave PIX
<code>src/app/(padrinho)/meus-aumigos/page.tsx</code>	260	Cartão por animal, novidades e notificações

Administração da ONG

ARQUIVO	LINHAS	PAPEL
<code>src/app/(ong)/adocoes/Acoes.tsx</code>	65	Só o próximo passo legítimo do funil
<code>src/app/(ong)/adocoes/actions.ts</code>	104	Status da adoção e o reflexo no status do animal
<code>src/app/(ong)/adocoes/page.tsx</code>	124	Funil em andamento e processos encerrados
<code>src/app/(ong)/alimentacao/Formularios.tsx</code>	190	Movimento e cadastro de item
<code>src/app/(ong)/alimentacao/actions.ts</code>	133	Saldo como consequência do movimento; entrada vira despesa
<code>src/app/(ong)/alimentacao/page.tsx</code>	193	Estoque com medidores, previsão e movimentos
<code>src/app/(ong)/animais-ong/FormularioAnimal.tsx</code>	345	Formulário em quatro blocos, com o custo somando ao vivo
<code>src/app/(ong)/animais-ong/[id]/PainelAcoes.tsx</code>	339	As quatro ações do dia a dia, em abas
<code>src/app/(ong)/animais-ong/[id]/editar/page.tsx</code>	52	Edição — o slug não muda junto com o nome
<code>src/app/(ong)/animais-ong/[id]/page.tsx</code>	362	Ficha completa: custo, padrinhos, saúde, diário, despesas
<code>src/app/(ong)/animais-ong/actions.ts</code>	379	Criar, editar, evento de diário, atualização e foto
<code>src/app/(ong)/animais-ong/novo/page.tsx</code>	21	Cadastro de animal
<code>src/app/(ong)/animais-ong/page.tsx</code>	204	Grade de animais com busca e cinco filtros

ARQUIVO	LINHAS	PAPEL
src/app/(ong)/apadrinhamentos/Acoes.tsx	76	Pausar com um clique; cancelar exige motivo
src/app/(ong)/apadrinhamentos/actions.ts	121	Mudança de status e alteração de valor, com auditoria
src/app/(ong)/apadrinhamentos/page.tsx	138	Receita recorrente, ticket médio e a lista por status
src/app/(ong)/campanhas-ong/Formularios.tsx	136	Nova campanha e as transições de status
src/app/(ong)/campanhas-ong/actions.ts	110	criarCampanha() e mudarStatusCampanha()
src/app/(ong)/campanhas-ong/page.tsx	129	Criação e ciclo de vida das campanhas
src/app/(ong)/configuracoes/Formularios.tsx	242	Quatro formulários da configuração
src/app/(ong)/configuracoes/actions.ts	166	Configuração, usuários e veterinários
src/app/(ong)/configuracoes/page.tsx	130	Dados da ONG, acessos, veterinários e auditoria
src/app/(ong)/financeiro/Painel.tsx	219	Confirmar pagamento, rotina mensal e lançamento de despesa
src/app/(ong)/financeiro/actions.ts	233	Confirmação, geração idempotente e lançamento com comprovante
src/app/(ong)/financeiro/page.tsx	279	Entradas, saídas, gráficos e a rotina do mês
src/app/(ong)/ia/Chat.tsx	174	Conversa com seis perguntas prontas e botão de copiar
src/app/(ong)/ia/page.tsx	22	Tela do assistente; degrada sem chave
src/app/(ong)/layout.tsx	37	Moldura da administração, com menu por papel
src/app/(ong)/padrinhos/[id]/page.tsx	227	Ficha: cadastro, contribuição, conquistas e histórico
src/app/(ong)/padrinhos/page.tsx	129	Lista com busca e receita recorrente
src/app/(ong)/painel/page.tsx	299	Dez indicadores, alertas acionáveis, quatro gráficos e agenda
src/app/(ong)/relatorios/page.tsx	191	Cobertura, inadimplência, retenção e descobertos
src/app/(ong)/saude/actions.ts	141	registrarSaude(): despesa, peso, castração e diário
src/app/(ong)/saude/page.tsx	189	Histórico, agenda de 30 dias e concentração de custo

API

ARQUIVO	LINHAS	PAPEL
<code>src/app/api/comprovante/[id]/route.ts</code>	41	Comprovante: público ou só para a equipe; 404 quando nega
<code>src/app/api/foto/[id]/route.ts</code>	30	Foto e vídeo do animal, público, com cache imutável
<code>src/app/api/health/route.ts</code>	19	Healthcheck que bate no banco
<code>src/app/api/ia/route.ts</code>	35	Recebe o histórico e devolve a resposta do modelo

Raiz e configuração

ARQUIVO	LINHAS	PAPEL
<code>.env.example</code>	51	Modelo de configuração, comentado
<code>Dockerfile</code>	63	Build em três estágios; runner enxuto e não-root
<code>README.md</code>	154	Visão geral, decisões e como rodar
<code>docker-compose.yml</code>	66	Serviço, volume e a rede com subnet fixa
<code>docker-entrypoint.sh</code>	21	Migrações, GRANTs e semente — nessa ordem, com o papel dono
<code>docs/OPERACAO.md</code>	103	Notas de quem mantém isto rodando
<code>next.config.mjs</code>	40	Saída standalone, cabeçalhos e indexação seletiva
<code>src/app/globals.css</code>	123	Oito classes de componente e a regra de prefers-reduced-motion
<code>src/app/layout.tsx</code>	33	Layout raiz: metadados, viewport, ícones
<code>tailwind.config.ts</code>	117	Paleta amostrada da logo, curvas e animações

Utilitários

ARQUIVO	LINHAS	PAPEL
<code>scripts/criar-usuario.ts</code>	55	Cria ou redefine acesso pela linha de comando; reativa a conta junto

B Comandos

B.1 Container

```
docker compose build      # reconstruir a imagem
docker compose up -d      # subir ou recriar
docker compose logs -f app # acompanhar
docker compose ps         # estado e saúde
docker compose down       # parar (o volume permanece)
```

B.2 Desenvolvimento

```
npm run dev          # servidor de desenvolvimento
npm run typecheck    # tsc --noEmit
npm run build        # prisma generate + next build
npm run db:seed      # semente (respeita as salvaguardas)
```

B.3 Banco

```
npm run prisma:migrate    # criar migração em desenvolvimento
npm run prisma:deploy     # aplicar migrações pendentes
npx prisma studio        # inspecionar dados
npx prisma migrate diff --from-empty \
  --to-schema-datamodel prisma/schema.prisma --script # SQL do schema inteiro
```

B.4 Emergência

```
docker compose exec app node_modules/.bin/tsx scripts/criar-usuario.ts \
  "Nome da Pessoa" email@ong.com.br senhaSegura ADMIN

curl -s localhost:1046/api/health          # o banco responde?
ufw status | grep 172.16.61                # o firewall está liberado?
docker compose exec app printenv DATABASE_URL
```

C Limitações conhecidas e próximos passos

Nada aqui é bug: são fronteiras conscientes do que foi construído. A ordem é de prioridade sugerida.

ITEM	IMPACTO	SITUAÇÃO E CAMINHO
Envio real de notificação	Alto	As notificações são gravadas e aparecem na área do padrinho, mas ninguém recebe e-mail. Como quem acompanha renova, este é o item que mais afeta a receita recorrente. Caminho: provedor transacional, domínio verificado e um worker que consome as notificações não enviadas.
Conciliação de pagamento	Alto	A confirmação é manual, uma a uma. Com PIX via API do banco ou um provedor de recorrência, a confirmação passaria a ser automática — e o campo <code>referencia</code> já existe na tabela justamente para a conciliação.
Geração mensal por agendador	Médio	Hoje é um botão. A ação já é idempotente, então basta um cron chamando-a — a parte difícil já está feita.
Rate limit nas rotas públicas de escrita	Médio	Apadrinhar, doar e manifestar interesse não têm limite por IP. A contenção natural é o Cloudflare, que já está no caminho da requisição.
Compressão de imagem no upload	Médio	A foto é gravada como veio, até 25 MB. Uma foto de celular moderno pesa 6 MB e é servida assim para quem abre no 4G. Caminho: <code>sharp</code> no servidor, gerando duas versões.

ITEM	IMPACTO	SITUAÇÃO E CAMINHO
Testes automatizados	Médio	<code>vitest</code> está configurado, mas não há suíte. Os candidatos naturais são as funções puras de <code>src/server/custos.ts</code> (o rateio, sobretudo, pelo arredondamento) e <code>src/lib/format.ts</code> .
Exclusão de foto pela interface	Baixo	Upload e definição de capa existem; remover exige acesso ao banco. <code>apagar()</code> já está em <code>storage.ts</code> .
Recuperação de senha	Baixo	Depende do envio de e-mail. Hoje o caminho é o admin redefinir pela tela de configurações ou pelo script.
Exportação de relatório	Baixo	Os relatórios são de tela. Um PDF de prestação de contas mensal para enviar a parceiros seria o próximo uso — e a página de impacto já tem todo o conteúdo.
Paginação	Baixo	A vitrine carrega todos os animais ativos e o financeiro traz 120 lançamentos. Na escala atual é instantâneo; acima de uns 500 animais, vale paginar.

D Glossário

TERMO	SIGNIFICADO NESTE SISTEMA
Apadrinhamento	Vínculo recorrente entre um padrinho e um animal, com valor mensal. Pode ser pausado, cancelado e retomado.
Competência	O mês que uma contribuição cobre, sempre representado pelo dia 1º. Diferente da data de pagamento.
Contribuição	Qualquer entrada de dinheiro: mensalidade, doação avulsa ou aporte de campanha.
Cobertura	Percentual do custo mensal estimado de um animal que já está coberto por apadrinhamentos ativos.
Custo estimado	Alimentação + tratamento + medicamento, definidos pela equipe. É a meta pública, não o gasto real.
Descoberto	A diferença entre o custo estimado e o apoio mensal — o quanto “falta” para o animal.
Diário	A biografia pública do animal, em linha do tempo. Distinto da atualização, que é a carta ao padrinho.
Modalidade	Faixa de apadrinhamento: alimentação (R\$ 30), cuidados (R\$ 50), completo (R\$ 100) ou personalizado.
Rateio	A divisão do valor pago por um padrinho entre as categorias de gasto daquele animal, no mês.
Retenção	Saldo entre apadrinhamentos novos e cancelados no período. O indicador de longo prazo mais importante.
Semente	O script que popula o banco com dados fictícios, com trava contra sobrescrever dado real.

TERMO	SIGNIFICADO NESTE SISTEMA
Slug	O endereço público do animal ou da campanha. Estável: não muda quando o nome muda.
Server Action	Função de servidor chamada diretamente por um formulário, sem endpoint REST. Onde vivem todas as mutações.
Server Component	Componente React que roda só no servidor, consulta o banco e transmite HTML. O padrão neste sistema.

SOBRE ESTE DOCUMENTO

Os trechos de código aqui não foram digitados à mão: são extraídos do código-fonte por um script no momento da geração. Se o código mudar e o documento for gerado de novo, os trechos acompanham. As contagens de linhas, arquivos e registros também vêm de medições reais feitas na geração, não de estimativas.